```
// 软件名称：墨译漫画小说智能翻译软件
// 版本号：V1.1.0
// 著作权人：薛朝坤
// 本文件为计算机软件著作权登记源程序鉴别材料（前30页+后30页）。
//
// ==================== 文件：commons/Index.ets ====================
/**
 * commons HAR entry.
 */
export { ThemeTokens } from './src/main/ets/ThemeTokens';
export { Log } from './src/main/ets/Log';
export { naturalSort, naturalCompare } from './src/main/ets/SortUtil';
export {
  WindowSizeClass,
  DeviceClass,
  sizeClassOfWidth,
  currentDeviceClass,
  screenWidthVp,
  startBreakpointObserver,
  observeBreakpoint
} from './src/main/ets/Breakpoint';export { toMediaUri, isUri } from './src/main/ets/PathUtil';
// ==================== 文件：commons/src/main/ets/Breakpoint.ets ====================
import { display, mediaquery, window } from '@kit.ArkUI';
import { deviceInfo } from '@kit.BasicServicesKit';
import { common } from '@kit.AbilityKit';

/**
 * One-time-development, multi-device sizing (一次开发，多端部署).
 * Official breakpoints: sm < 600vp, md 600–839vp, lg >= 840vp.
 */
export enum WindowSizeClass {
  SM = 0,
  MD = 1,
  LG = 2
}

/** Coarse hardware class used to tailor interaction (hover cursor etc.). */
export enum DeviceClass {
  PHONE = 0,
  TABLET = 1,
  PC = 2,
  OTHER = 3
}

export function sizeClassOfWidth(vp: number): WindowSizeClass {
  if (vp >= 840) {
    return WindowSizeClass.LG;
  }
  if (vp >= 600) {
    return WindowSizeClass.MD;
```

```
  }
  return WindowSizeClass.SM;
}

export function currentDeviceClass(): DeviceClass {
  const t = deviceInfo.deviceType;
  if (t === '2in1' || t === 'pc') {
    return DeviceClass.PC;
  }
  if (t === 'tablet') {
    return DeviceClass.TABLET;
  }
  if (t === 'phone' || t === 'default') {
    return DeviceClass.PHONE;
  }
  return DeviceClass.OTHER;
}

/** Screen width in vp (fallback / initial value before layout). */
export function screenWidthVp(): number {
  const d = display.getDefaultDisplaySync();
  const density = d.densityPixels > 0 ? d.densityPixels : 1;
  return d.width / density;
}

function vpWidthOfScreen(): number {
  return screenWidthVp();
}

/**
 * Reliable breakpoint observer for the app shell:
 * initial value is derived from the real display/window size (media queries can
 * report a pre-layout width on tablets), then updated on window resize.
 * Must be called from a UIAbility/UI page context.
 */
export async function startBreakpointObserver(context: common.Context,
  onChange: (c: WindowSizeClass) => void): Promise<() => void> {
  let win: window.Window | undefined = undefined;
  try {
    win = await window.getLastWindow(context);
  } catch (e) {
    win = undefined;
  }
  const apply = () => {
    let vp = vpWidthOfScreen();
    if (win !== undefined) {
      try {
        const rect = win.getWindowProperties().windowRect;
        const density = display.getDefaultDisplaySync().densityPixels;
        if (rect.width > 0 && density > 0) {
```

```
          vp = rect.width / density;
        }
      } catch (e) {
        // keep screen-derived fallback
      }
    }
    onChange(sizeClassOfWidth(vp));
  };
  apply();
  const sizeCb = (size: window.Size) => {
    const density = display.getDefaultDisplaySync().densityPixels;
    const vp = density > 0 ? size.width / density : size.width;
    onChange(sizeClassOfWidth(vp));
  };
  if (win !== undefined) {
    try {
      win.on('windowSizeChange', sizeCb);
    } catch (e) {
      // ignore and rely on media queries below
    }
  }
  const smListener = mediaquery.matchMediaSync('(width>=600vp)');
  const lgListener = mediaquery.matchMediaSync('(width>=840vp)');
  const emit = () => {
    let c = WindowSizeClass.SM;
    if (lgListener.matches) {
      c = WindowSizeClass.LG;
    } else if (smListener.matches) {
      c = WindowSizeClass.MD;
    }
    onChange(c);
  };
  smListener.on('change', emit);
  lgListener.on('change', emit);
  return () => {
    if (win !== undefined) {
      try {
        win.off('windowSizeChange', sizeCb);
      } catch (e) {
        // ignore
      }
    }
    smListener.off('change', emit);
    lgListener.off('change', emit);
  };
}

/**
 * Lightweight media-query observer (page level). Returns a stop function.
 */
```

```
export function observeBreakpoint(onChange: (c: WindowSizeClass) => void): () => void {
  const smListener = mediaquery.matchMediaSync('(width>=600vp)');
  const lgListener = mediaquery.matchMediaSync('(width>=840vp)');
  const emit = () => {
    let c = WindowSizeClass.SM;
    if (lgListener.matches) {
      c = WindowSizeClass.LG;
    } else if (smListener.matches) {
      c = WindowSizeClass.MD;
    }
    onChange(c);
  };
  smListener.on('change', emit);
  lgListener.on('change', emit);
  emit();
  return () => {
    smListener.off('change', emit);
    lgListener.off('change', emit);
  };
}
// ==================== 文件: commons/src/main/ets/Log.ets ====================
import { hilog } from '@kit.PerformanceAnalysisKit';

/**
 * Thin hilog wrapper with a fixed domain and per-call tags.
 */
export class Log {
  private static readonly DOMAIN: number = 0x0301;

  static debug(tag: string, msg: string): void {
    hilog.debug(Log.DOMAIN, tag, '%{public}s', msg);
  }

  static info(tag: string, msg: string): void {
    hilog.info(Log.DOMAIN, tag, '%{public}s', msg);
  }

  static warn(tag: string, msg: string): void {
    hilog.warn(Log.DOMAIN, tag, '%{public}s', msg);
  }

  static error(tag: string, msg: string): void {
    hilog.error(Log.DOMAIN, tag, '%{public}s', msg);
  }
}
// ==================== 文件: commons/src/main/ets/PathUtil.ets ====================
import { fileUri } from '@kit.CoreFileKit';

/**
 * Media path helpers.
```

```
 *
 * ArkUI's Image component does NOT accept a raw sandbox path like
 * /data/storage/el2/base/haps/entry/files/xxx.png — it needs a sandbox URI
 * (file://<bundle>/<sandboxPath>) or a PixelMap. Imported pages, covers and
 * exported previews are stored as sandbox paths, so every Image source and
 * ImageSource creation should pass through toMediaUri().
 */
export function toMediaUri(pathOrUri: string): string {
  if (pathOrUri.length === 0) {
    return pathOrUri;
  }
  if (pathOrUri.startsWith('file://') || pathOrUri.startsWith('http://') ||
    pathOrUri.startsWith('https://') || pathOrUri.startsWith('datashare://') ||
    pathOrUri.startsWith('content://') || pathOrUri.startsWith('ohos://') ||
    pathOrUri.startsWith('$')) {
    return pathOrUri;
  }
  if (pathOrUri.startsWith('/')) {
    try {
      return fileUri.getUriFromPath(pathOrUri);
    } catch (e) {
      return pathOrUri;
    }
  }
  return pathOrUri;
}

/** True when the value is already a URI accepted by Image / ImageSource. */
export function isUri(value: string): boolean {
  return value.startsWith('file://') || value.startsWith('http://') ||
value.startsWith('https://') ||
    value.startsWith('datashare://') || value.startsWith('content://');
}
// ==================== 文件: commons/src/main/ets/SortUtil.ets ====================
/**
 * Digit-aware natural ordering so page names sort v2 < v10 (plan D10).
 */
function isDigit(c: string): boolean {
  return c >= '0' && c <= '9';
}

/**
 * Compare two strings naturally: digit runs compare numerically,
 * other characters compare by code point. Returns <0 / 0 / >0.
 */
export function naturalCompare(a: string, b: string): number {
  let i = 0;
  let j = 0;
  while (i < a.length && j < b.length) {
    const ca = a.charAt(i);
```

```
    const cb = b.charAt(j);
    if (isDigit(ca) && isDigit(cb)) {
      let ai = i;
      let bj = j;
      while (ai < a.length && isDigit(a.charAt(ai))) {
        ai++;
      }
      while (bj < b.length && isDigit(b.charAt(bj))) {
        bj++;
      }
      const rawA = a.substring(i, ai);
      const rawB = b.substring(j, bj);
      let za = 0;
      while (za < rawA.length - 1 && rawA.charAt(za) === '0') {
        za++;
      }
      let zb = 0;
      while (zb < rawB.length - 1 && rawB.charAt(zb) === '0') {
        zb++;
      }
      const tA = rawA.substring(za);
      const tB = rawB.substring(zb);
      if (tA.length !== tB.length) {
        return tA.length - tB.length;
      }
      if (tA !== tB) {
        return tA < tB ? -1 : 1;
      }
      if (rawA.length !== rawB.length) {
        return rawA.length - rawB.length;
      }
      i = ai;
      j = bj;
    } else {
      const codeA = a.codePointAt(i) ?? 0;
      const codeB = b.codePointAt(j) ?? 0;
      if (codeA !== codeB) {
        return codeA - codeB;
      }
      i++;
      j++;
    }
  }
  const remA = a.length - i;
  const remB = b.length - j;
  if (remA === remB) {
    return 0;
  }
  return remA < remB ? -1 : 1;
}
```

```
/** Return a NEW naturally-sorted array (input untouched). */
export function naturalSort(names: Array<string>): Array<string> {
  const copy: Array<string> = names.slice(0, names.length);
  copy.sort((a: string, b: string): number => naturalCompare(a, b));
  return copy;
}
// ==================== 文件: commons/src/main/ets/ThemeTokens.ets ====================
/**
 * Moyi HarmonyOS design tokens — 蓝白 / 蓝黑（默认蓝白）.
 * 已移除紫色主题；token 名称保持不变，页面无需改动。
 */
export type ThemeMode = 'blue' | 'blueDark';

export class ThemeTokens {
  private static modeValue: ThemeMode = 'blue';

  static SURFACE: string = '#FFFFFF';
  static SURFACE_VARIANT: string = '#F2F7FF';
  static SURFACE_SECONDARY: string = '#E9F2FF';
  static SURFACE_HIGH: string = '#DCE9FB';
  static CARD_BG: string = '#FFFFFF';
  static CARD_HOVER: string = '#F5FAFF';
  static DIVIDER: string = '#DCE8F7';
  static SCRIM: string = '#CC0B1220';

  static TEXT_PRIMARY: string = '#0B1F33';
  static TEXT_SECONDARY: string = '#3E566F';
  static TEXT_HINT: string = '#6C8299';
  static TEXT_TERTIARY: string = '#93A7BC';
  static TEXT_ON_PRIMARY: string = '#FFFFFF';

  static PRIMARY: string = '#0A66FF';
  static PRIMARY_DEEP: string = '#004FCC';
  static PRIMARY_SOFT: string = '#5B97FF';
  static ACCENT: string = '#2F7CFF';
  static RED: string = '#D93B4F';
  static SUCCESS: string = '#12805C';
  static WARN: string = '#B26A00';
  static WHITE: string = '#FFFFFF';
  static BLACK: string = '#0A1220';

  static MATERIAL_TINT: string = '#CCFFFFFF';
  static GLASS_BORDER: string = '#661F6FEB';

  static readonly READER_BG_BLACK: string = '#000000';
  static readonly READER_BG_NOCTURNE: string = '#0B0813';
  static readonly READER_BG_PLUM: string = '#16112A';
  static readonly READER_BG_DEEP: string = '#221A38';
  static readonly READER_BG_PAPER: string = '#F5EFE4';
```

```
  static readonly READER_BG_WHITE: string = '#FFFFFF';
  static readonly READER_BG_GRAY: string = '#808080';

  static readonly FONT_TITLE: number = 20;
  static readonly FONT_SUBTITLE: number = 17;
  static readonly FONT_BODY: number = 16;
  static readonly FONT_CAPTION: number = 13;
  static readonly FONT_SMALL: number = 12;
  static readonly FONT_DISPLAY: number = 28;

  static readonly RADIUS_CARD: number = 16;
  static readonly RADIUS_SMALL: number = 10;
  static readonly RADIUS_PILL: number = 999;

  static readonly PAGE_PADDING: number = 16;
  static readonly CARD_PADDING: number = 14;
  static readonly GAP: number = 12;

  static readonly DURATION_FAST: number = 150;
  static readonly DURATION_BASE: number = 240;

  /** 蓝白/蓝黑运行时切换；未知旧值（purple/oled/system）迁移到蓝白。 */
  static applyMode(mode: string): void {
    if (mode === 'blueDark') {
      ThemeTokens.modeValue = 'blueDark';
      ThemeTokens.SURFACE = '#071426';
      ThemeTokens.SURFACE_VARIANT = '#0D1F35';
      ThemeTokens.SURFACE_SECONDARY = '#12283F';
      ThemeTokens.SURFACE_HIGH = '#18334F';
      ThemeTokens.CARD_BG = '#0D1F35';
      ThemeTokens.CARD_HOVER = '#14304A';
      ThemeTokens.DIVIDER = '#1E3A57';
      ThemeTokens.SCRIM = '#E6000000';
      ThemeTokens.TEXT_PRIMARY = '#E8F2FF';
      ThemeTokens.TEXT_SECONDARY = '#A9C2DD';
      ThemeTokens.TEXT_HINT = '#7693B2';
      ThemeTokens.TEXT_TERTIARY = '#5A7693';
      ThemeTokens.TEXT_ON_PRIMARY = '#FFFFFF';
      ThemeTokens.PRIMARY = '#5B9CFF';
      ThemeTokens.PRIMARY_DEEP = '#3B7BDD';
      ThemeTokens.PRIMARY_SOFT = '#8DBCFF';
      ThemeTokens.ACCENT = '#6EA8FF';
      ThemeTokens.RED = '#FF7A8A';
      ThemeTokens.SUCCESS = '#4FD1A5';
      ThemeTokens.WARN = '#F0B95C';
      ThemeTokens.WHITE = '#FFFFFF';
      ThemeTokens.BLACK = '#000000';
      ThemeTokens.MATERIAL_TINT = '#33FFFFFF';
      ThemeTokens.GLASS_BORDER = '#3DFFFFFF';
      return;
```

```
    }
    ThemeTokens.modeValue = 'blue';
    ThemeTokens.SURFACE = '#FFFFFF';
    ThemeTokens.SURFACE_VARIANT = '#F2F7FF';
    ThemeTokens.SURFACE_SECONDARY = '#E9F2FF';
    ThemeTokens.SURFACE_HIGH = '#DCE9FB';
    ThemeTokens.CARD_BG = '#FFFFFF';
    ThemeTokens.CARD_HOVER = '#F5FAFF';
    ThemeTokens.DIVIDER = '#DCE8F7';
    ThemeTokens.SCRIM = '#CC0B1220';
    ThemeTokens.TEXT_PRIMARY = '#0B1F33';
    ThemeTokens.TEXT_SECONDARY = '#3E566F';
    ThemeTokens.TEXT_HINT = '#6C8299';
    ThemeTokens.TEXT_TERTIARY = '#93A7BC';
    ThemeTokens.TEXT_ON_PRIMARY = '#FFFFFF';
    ThemeTokens.PRIMARY = '#0A66FF';
    ThemeTokens.PRIMARY_DEEP = '#004FCC';
    ThemeTokens.PRIMARY_SOFT = '#5B97FF';
    ThemeTokens.ACCENT = '#2F7CFF';
    ThemeTokens.RED = '#D93B4F';
    ThemeTokens.SUCCESS = '#12805C';
    ThemeTokens.WARN = '#B26A00';
    ThemeTokens.WHITE = '#FFFFFF';
    ThemeTokens.BLACK = '#0A1220';
    ThemeTokens.MATERIAL_TINT = '#CCFFFFFF';
    ThemeTokens.GLASS_BORDER = '#661F6FEB';
  }

  static mode(): ThemeMode {
    return ThemeTokens.modeValue;
  }

  static isLight(): boolean {
    return ThemeTokens.modeValue === 'blue';
  }
}
// ==================== 文件: entry/src/main/cpp/types/libllamacpp/index.d.ets ====================
/**
 * NAPI contract for libllamacpp.so (plan D11).
 * generate() streams tokens through onToken and reports completion
 * (full text, or error message when isError) through onDone.
 */
export const getVersion: () => string;

export const loadModel: (modelPath: string) => number;

export const generate: (handle: number, prompt: string, maxTokens: number, temperature: number,
  onToken: (token: string) => void,
  onDone: (text: string, isError: boolean) => void) => void;
```

```
export const stopGeneration: (handle: number) => void;

export const unloadModel: (handle: number) => void;
// ==================== 文件: entry/src/main/cpp/types/libocrasync/index.d.ets ====================
export const initRec: (modelPath: string) => boolean;
export const initDet: (modelPath: string) => boolean;
export const detectBoxes: (rgba: ArrayBuffer, width: number, height: number) => Array<number>;
export const recognizeBoxes: (rgba: ArrayBuffer, width: number, height: number,
  boxes: Array<number>, dict: string) => Array<string>;
// ==================== 文件: entry/src/test/List.test.ets ====================
import { describe, it, expect } from '@ohos/hypium';
import { util } from '@kit.ArkTS';
import { naturalSort } from 'commons';
import {
  DisplayRect,
  FontSizeMode,
  LayoutEngine,
  RenderOptions,
  RenderOrientation,
  VerticalDirection,
  containFit,
  pixelToDisplay
} from 'corerender';
import { BingTranslator, SseParser } from 'coretranslate';
import { makePixelRect } from 'coremodel';

function toBuffer(bytes: Array<number>): ArrayBuffer {
  const arr = new Uint8Array(bytes.length);
  for (let i = 0; i < bytes.length; i++) {
    arr[i] = bytes[i] & 0xFF;
  }
  return arr.buffer;
}

function utf8Bytes(s: string): Array<number> {
  const encoded = new util.TextEncoder().encodeInto(s);
  const out: Array<number> = [];
  for (let i = 0; i < encoded.length; i++) {
    out.push(encoded[i]);
  }
  return out;
}

function renderOptions(orientation: RenderOrientation, mode: FontSizeMode, size: number):
RenderOptions {
  const o: RenderOptions = {
    orientation: orientation,
    verticalDirection: VerticalDirection.RTL,
    fontSizeMode: mode,
    fixedFontSize: size,
```

```
    padding: 4
  };
  return o;
}

function makeBox(w: number, h: number): DisplayRect {
  const box: DisplayRect = { x: 0, y: 0, w: w, h: h };
  return box;
}

export default function localUnitTest() {
  describe('Moyi local unit tests', () => {
    // ---------- T044: coordinate mapping ----------
    it('containFit_scales_and_centers', 0, () => {
      const fit = containFit(100, 200, 100, 100);
      expect(fit.scale).assertEqual(0.5);
      expect(fit.dx).assertEqual(25);
      expect(fit.dy).assertEqual(0);
    });

    it('pixelToDisplay_maps_box', 0, () => {
      const rect = pixelToDisplay(makePixelRect(10, 20, 30, 40), 0.5, 5, 5);
      expect(rect.x).assertEqual(10);
      expect(rect.y).assertEqual(15);
      expect(rect.w).assertEqual(15);
      expect(rect.h).assertEqual(20);
    });

    // ---------- T044: natural sort (v2 < v10) ----------
    it('naturalSort_numeric_aware', 0, () => {
      const sorted = naturalSort(['v10.jpg', 'v2.jpg', 'v1.jpg', 'v20.jpg']);
      expect(sorted[0]).assertEqual('v1.jpg');
      expect(sorted[1]).assertEqual('v2.jpg');
      expect(sorted[2]).assertEqual('v10.jpg');
      expect(sorted[3]).assertEqual('v20.jpg');
    });

    it('naturalSort_mixed_names', 0, () => {
      const sorted = naturalSort(['page-2.png', 'page-10.png', 'cover.png']);
      expect(sorted[0]).assertEqual('cover.png');
      expect(sorted[1]).assertEqual('page-2.png');
      expect(sorted[2]).assertEqual('page-10.png');
    });

    // ---------- T044: layout / ellipsis / font auto ----------
    it('layout_horizontal_auto_font_nonzero', 0, () => {
      const layout = LayoutEngine.layout('好', makeBox(120, 120),
        renderOptions(RenderOrientation.HORIZONTAL, FontSizeMode.AUTO, 16));
      expect(layout.runs.length).assertEqual(1);
      expect(layout.measuredFontSize >= 10).assertTrue();
```

```
});

it('layout_vertical_runs_per_char', 0, () => {
  const layout = LayoutEngine.layout('你好世界', makeBox(80, 240),
    renderOptions(RenderOrientation.VERTICAL, FontSizeMode.AUTO, 16));
  expect(layout.runs.length).assertEqual(4);
});

it('layout_fixed_font_overflow_truncates_with_ellipsis', 0, () => {
  const opts = renderOptions(RenderOrientation.HORIZONTAL, FontSizeMode.FIXED, 24);
  const layout = LayoutEngine.layout('这是一段很长很长的译文需要被截断处理不能拆行', makeBox(40, 28), opts);
  expect(layout.truncated).assertTrue();
  let last = '';
  if (layout.runs.length > 0) {
    last = layout.runs[layout.runs.length - 1].char;
  }
  expect(last).assertEqual('…');
});

it('layout_vertical_rtl_column_order', 0, () => {
  const opts = renderOptions(RenderOrientation.VERTICAL, FontSizeMode.FIXED, 20);
  const layout = LayoutEngine.layout('你好世界一二', makeBox(40, 60), opts);
  expect(layout.runs.length > 0).assertTrue();
});

// ---------- T045: SSE parsing ----------
it('sse_single_event_and_done', 0, () => {
  const events: Array<string> = [];
  let done = false;
  const parser = new SseParser({
    onEvent: (data: string) => {
      events.push(data);
    },
    onDone: () => {
      done = true;
    }
  });
  parser.push(toBuffer(utf8Bytes('data: hello\n\n')));
  parser.push(toBuffer(utf8Bytes('data: [DONE]\n\n')));
  expect(events.length).assertEqual(1);
  expect(events[0]).assertEqual('hello');
  expect(done).assertTrue();
});

it('sse_multi_event_framing', 0, () => {
  const events: Array<string> = [];
  const parser = new SseParser({
    onEvent: (data: string) => {
      events.push(data);
    },
```

```
        onDone: () => {
        }
      });
      parser.push(toBuffer(utf8Bytes('data: a\n\ndata: b\n\n')));
      expect(events.length).assertEqual(2);
      expect(events[0]).assertEqual('a');
      expect(events[1]).assertEqual('b');
    });

    it('sse_cross_chunk_utf8_safe', 0, () => {
      const events: Array<string> = [];
      const parser = new SseParser({
        onEvent: (data: string) => {
          events.push(data);
        },
        onDone: () => {
        }
      });
      // "data: 日\n\n" — 日 = E6 97 A5, split so the 3 bytes span two chunks.
      const bytes = utf8Bytes('data: 日\n\n');
      const split = 7;
      parser.push(toBuffer(bytes.slice(0, split)));
      parser.push(toBuffer(bytes.slice(split)));
      expect(events.length).assertEqual(1);
      expect(events[0]).assertEqual('日');
    });

    // ---------- T045: Bing request assembly ----------
    it('bing_form_body_shape', 0, () => {
      const body = BingTranslator.buildFormBody('zh', 'en', '你好', 'tok123', 'key456');
      expect(body.indexOf('fromLang=zh-Hans') >= 0).assertTrue();
      expect(body.indexOf('to=en') >= 0).assertTrue();
      expect(body.indexOf(`text=${encodeURIComponent('你好')}`) >= 0).assertTrue();
      expect(body.indexOf('token=tok123') >= 0).assertTrue();
      expect(body.indexOf('key=key456') >= 0).assertTrue();
    });
  });
}
// ==================== 文件: entry/src/main/ets/entryability/EntryAbility.ets ====================
/*
 * Copyright (c) 2026 MoyiOhos contributors.
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://www.apache.org/licenses/LICENSE-2.0
 */

import { AbilityConstant, ConfigurationConstant, UIAbility, Want } from '@kit.AbilityKit';
import { window } from '@kit.ArkUI';
```

```
import { preferences } from '@kit.ArkData';
import { BusinessError } from '@kit.BasicServicesKit';
import { AppEnv } from '../common/AppEnv';
import { OcrService } from 'coreocr';
import { PageTranslateService } from '../service/PageTranslateService';
import { Log, ThemeTokens } from 'commons';

/** App shell: persisted palette (蓝白/蓝黑) is applied before loadContent; 旧紫色值迁移到蓝白。 */
export default class EntryAbility extends UIAbility {
  onCreate(want: Want, launchParam: AbilityConstant.LaunchParam): void {
    let mode = 'blue';
    try {
      const store = preferences.getPreferencesSync(this.context, { name: 'starflow_settings' });
      const saved = store.getSync('theme_mode', 'blue');
      mode = saved as string;
      mode = mode === 'blueDark' ? 'blueDark' : 'blue';
    } catch (e) {
      Log.warn('EntryAbility', 'theme preference read failed: ' + JSON.stringify(e));
    }
    ThemeTokens.applyMode(mode);
    try {
      this.context.getApplicationContext().setColorMode(mode === 'blue'
        ? ConfigurationConstant.ColorMode.COLOR_MODE_LIGHT
        : ConfigurationConstant.ColorMode.COLOR_MODE_DARK);
    } catch (err) {
      Log.error('EntryAbility', `Failed to set colorMode. Cause: ${JSON.stringify(err)}`);
    }
    // Application-wide assembly: context, RDB init, engine registry,
    // local model path restore (plan: common/AppEnv.ets). Fire-and-forget;
    // pages await AppEnv.ready() before touching the database.
    AppEnv.init(this.context).catch((e: Object) => {
      Log.error('EntryAbility', `AppEnv init failed: ${e}`);
    });
    Log.info('EntryAbility', 'Ability onCreate');
  }

  onDestroy(): void {
    Log.info('EntryAbility', 'Ability onDestroy');
  }

  onWindowStageCreate(windowStage: window.WindowStage): void {
    Log.info('EntryAbility', 'Ability onWindowStageCreate');
    windowStage.loadContent('pages/Index', (err) => {
      if (err.code) {
        Log.error('EntryAbility', `Failed to load the content. Cause: ${JSON.stringify(err)}`);
        return;
      }
      Log.info('EntryAbility', 'Succeeded in loading the content.');
      this.configureWindow(windowStage);
      // 系统 OCR 首次 init 在真机上可能耗时数十秒；启动后后台预热，避免首张漫画翻译被冷启动拖到 1 分钟。
```

```
    setTimeout(() => {
      OcrService.getInstance().ensureInit().catch((e: Error) => {
        Log.warn('EntryAbility', 'OCR prewarm skipped: ' + e.message);
      });
      PageTranslateService.prewarm().catch((e: Error) => {
        Log.warn('EntryAbility', 'local OCR/model prewarm skipped: ' + e.message);
      });
    }, 1500);
    // 本地 GGUF 首次加载 1–3s；启动后预热，避免首句翻译超时。
    setTimeout(() => {
      const translator = AppEnv.getInstance().getLocalTranslator();
      if (translator !== undefined) {
        translator.prewarm().catch((e: Error) => {
          Log.warn('EntryAbility', 'local llama prewarm skipped: ' + e.message);
        });
      }
    }, 3000);
  });
}

/**
 * System bar + window chrome: blue-white / blue-black background matching the theme.
 * Works for phone / tablet / 2in1 (window chrome is managed by the system on PC).
 */
private configureWindow(windowStage: window.WindowStage): void {
  try {
    const mainWindow: window.Window = windowStage.getMainWindowSync();
    mainWindow.setWindowLayoutFullScreen(true).catch((e: BusinessError) => {
      Log.warn('EntryAbility', `setWindowLayoutFullScreen failed: ${e.message}`);
    });
    mainWindow.setWindowBackgroundColor(ThemeTokens.SURFACE);
    mainWindow.setWindowSystemBarProperties({
      statusBarColor: ThemeTokens.SURFACE,
      statusBarContentColor: ThemeTokens.isLight() ? '#0F2742' : '#FFFFFF',
      navigationBarColor: ThemeTokens.SURFACE,
      navigationBarContentColor: ThemeTokens.isLight() ? '#0F2742' : '#FFFFFF'
    }).catch((e: BusinessError) => {
      Log.warn('EntryAbility', `setWindowSystemBarProperties failed: ${e.message}`);
    });
  } catch (e) {
    Log.warn('EntryAbility', `configureWindow failed: ${e}`);
  }
}

onWindowStageDestroy(): void {
  Log.info('EntryAbility', 'Ability onWindowStageDestroy');
}

onForeground(): void {
  Log.info('EntryAbility', 'Ability onForeground');
```

```
  }

  onBackground(): void {
    Log.info('EntryAbility', 'Ability onBackground');
  }
}
// ==================== 文件: entry/src/main/ets/pages/AboutPage.ets ====================
import { uiMaterial } from '@kit.ArkUI';
import { ThemeTokens, WindowSizeClass, observeBreakpoint } from 'commons';
import { AppEnv } from '../common/AppEnv';
import { BuildInfo } from '../common/BuildInfo';
import { Routes } from '../common/Routes';

/** 信息行。 */
class InfoRow {
  label: string;
  value: string;

  constructor(label: string, value: string) {
    this.label = label;
    this.value = value;
  }
}

/** 卡片容器。 */
@Component
struct AboutCard {
  @BuilderParam content: () => void;

  build() {
    Column() {
      this.content()
    }
    .width('100%')
    .padding(ThemeTokens.CARD_PADDING)
    .borderRadius(ThemeTokens.RADIUS_CARD)
    .backgroundColor(ThemeTokens.CARD_BG)
    .alignItems(HorizontalAlign.Start)
  }
}

/**
 * 关于页（S110）：品牌、版本、构建模块清单（1 HAP + 6 HAR）、技术栈、
 * 许可与来源、快捷入口（开源致谢 / 常见问题 / 重新查看引导）。
 */
@Component
export struct AboutPage {
  @State sizeClass: WindowSizeClass = WindowSizeClass.SM;
  private navStack: NavPathStack = AppEnv.getInstance().navStack;
  private stopBreakpoint: (() => void) | undefined = undefined;
```

```
  private rows: Array<InfoRow> = [
    new InfoRow('应用版本', `v${BuildInfo.APP_VERSION}`),
    new InfoRow('Bundle 名称', BuildInfo.BUNDLE_NAME),
    new InfoRow('API Level', `HarmonyOS ${BuildInfo.HARMONY_API_LEVEL}`),
    new InfoRow('构建日期', BuildInfo.BUILD_DATE),
    new InfoRow('设备形态', 'phone · tablet · 2in1')
  ];

  aboutToAppear(): void {
    this.stopBreakpoint = observeBreakpoint((c: WindowSizeClass) => {
      this.sizeClass = c;
    });
  }

  aboutToDisappear(): void {
    if (this.stopBreakpoint !== undefined) {
      this.stopBreakpoint();
      this.stopBreakpoint = undefined;
    }
  }

  private isWide(): boolean {
    return this.sizeClass === WindowSizeClass.LG;
  }

  @Builder
  sectionTitle(title: string) {
    Text(title)
      .fontSize(ThemeTokens.FONT_BODY)
      .fontWeight(FontWeight.Medium)
      .fontColor(ThemeTokens.TEXT_PRIMARY)
      .width('100%')
      .margin({ top: 6, bottom: 2 })
  }

  @Builder
  infoLine(row: InfoRow) {
    Row() {
      Text(row.label)
        .fontSize(ThemeTokens.FONT_SMALL)
        .fontColor(ThemeTokens.TEXT_SECONDARY)
      Blank()
      Text(row.value)
        .fontSize(ThemeTokens.FONT_SMALL)
        .fontColor(ThemeTokens.TEXT_PRIMARY)
        .maxLines(1)
        .textOverflow({ overflow: TextOverflow.Ellipsis })
    }
    .width('100%')
    .padding({ top: 7, bottom: 7 })
```

```
    .alignItems(VerticalAlign.Center)
  }

  @Builder
  moduleChip(name: string, hap: boolean) {
    Text(name)
      .fontSize(ThemeTokens.FONT_SMALL)
      .fontColor(hap ? ThemeTokens.TEXT_ON_PRIMARY : ThemeTokens.PRIMARY_SOFT)
      .padding({ left: 9, right: 9, top: 5, bottom: 5 })
      .borderRadius(ThemeTokens.RADIUS_PILL)
      .backgroundColor(hap ? ThemeTokens.PRIMARY : ThemeTokens.SURFACE_VARIANT)
      .margin({ right: 6, bottom: 6 })
  }

  @Builder
  hero() {
    Column({ space: 8 }) {
      Image($r('app.media.logo_starflow'))
        .width(72)
        .height(72)
        .borderRadius(20)
      Text('墨译 Moyi')
        .fontSize(ThemeTokens.FONT_DISPLAY)
        .fontWeight(FontWeight.Bold)
        .fontColor(ThemeTokens.TEXT_PRIMARY)
      Text('漫画 · 小说 · 文本翻译 · 本地模型')
        .fontSize(ThemeTokens.FONT_CAPTION)
        .fontColor(ThemeTokens.TEXT_SECONDARY)
      Row({ space: 8 }) {
        Text(`v${BuildInfo.APP_VERSION}`)
          .fontSize(ThemeTokens.FONT_SMALL)
          .fontColor(ThemeTokens.PRIMARY)
          .padding({ left: 9, right: 9, top: 3, bottom: 3 })
          .borderRadius(ThemeTokens.RADIUS_PILL)
          .backgroundColor(ThemeTokens.SURFACE_VARIANT)
        Text('HarmonyOS · 蓝白阅读')
          .fontSize(ThemeTokens.FONT_SMALL)
          .fontColor(ThemeTokens.TEXT_HINT)
          .padding({ left: 9, right: 9, top: 3, bottom: 3 })
          .borderRadius(ThemeTokens.RADIUS_PILL)
          .backgroundColor(ThemeTokens.SURFACE_VARIANT)
      }
    }
    .width('100%')
    .padding({ top: 14, bottom: 10 })
    .alignItems(HorizontalAlign.Center)
    .justifyContent(FlexAlign.Center)
  }

  build() {
```

```
    NavDestination() {
      Scroll() {
        Row() {
          Column({ space: 12 }) {
            this.hero()

            this.sectionTitle('版本与构建')
            AboutCard() {
              ForEach(this.rows, (row: InfoRow, index: number) => {
                if (index > 0) {
                  Divider()
                    .color(ThemeTokens.DIVIDER)
                    .strokeWidth(0.5)
                }
                this.infoLine(row)
              }, (row: InfoRow) => row.label)
            }

            this.sectionTitle(`构建模块（1 HAP + ${BuildInfo.MODULES.length - 1} HAR）`)
            AboutCard() {
              Flex({ wrap: FlexWrap.Wrap }) {
                ForEach(BuildInfo.MODULES, (m: string) => {
                  this.moduleChip(m, m.indexOf('(HAP)') >= 0)
                }, (m: string) => m)
              }
              .width('100%')
              Text(`共 ${BuildInfo.MODULE_COUNT} 个模块；entry 为 HAP，其余为可复用 HAR。构建产物由
devecocli（DevEco 26 / API 26）生成。`)
                .fontSize(ThemeTokens.FONT_SMALL)
                .fontColor(ThemeTokens.TEXT_HINT)
                .width('100%')
                .lineHeight(17)
                .margin({ top: 2 })
            }

            this.sectionTitle('技术栈')
            AboutCard() {
              Text('ArkTS + ArkUI 声明式开发（V1 状态管理）· 关系型数据库 RDB · Preferences 设置持久化 · Asset Store
Kit 密钥存储 · uiMaterial 沉浸光感（API 26）· Core Vision Kit 文字识别 · llama.cpp NAPI 端侧推理。')
                .fontSize(ThemeTokens.FONT_SMALL)
                .fontColor(ThemeTokens.TEXT_SECONDARY)
                .width('100%')
                .lineHeight(19)
            }

            this.sectionTitle('许可与来源')
            AboutCard() {
              Text('本应用参照上游开源项目 StarFlow（星译，LGPL-3.0）的信息架构与行为，在 HarmonyOS 上以 ArkTS
独立重写实现，未直接复制上游代码文本。')
                .fontSize(ThemeTokens.FONT_SMALL)
```

```
        .fontColor(ThemeTokens.TEXT_SECONDARY)
        .width('100%')
        .lineHeight(19)
      Divider()
        .color(ThemeTokens.DIVIDER)
        .strokeWidth(0.5)
        .margin({ top: 8, bottom: 8 })
      Text('llama.cpp（MIT）用于本地 GGUF 推理；OCR 能力来自 HarmonyOS Core Vision Kit；本地模型 Hy-MT2
的版权与许可归腾讯混元。完整列表见「开源致谢」。')
        .fontSize(ThemeTokens.FONT_SMALL)
        .fontColor(ThemeTokens.TEXT_SECONDARY)
        .width('100%')
        .lineHeight(19)
      Text('数据默认保存在本机；应用不含广告、内购与统计 SDK，也无账号体系。')
        .fontSize(ThemeTokens.FONT_SMALL)
        .fontColor(ThemeTokens.TEXT_HINT)
        .width('100%')
        .lineHeight(19)
        .margin({ top: 8 })
    }

    Row({ space: 10 }) {
      Button('开源致谢')
        .fontSize(ThemeTokens.FONT_SMALL)
        .fontColor(ThemeTokens.TEXT_ON_PRIMARY)
        .backgroundColor(ThemeTokens.PRIMARY)
        .borderRadius(ThemeTokens.RADIUS_PILL)
        .height(36)
        .layoutWeight(1)
        .onClick(() => {
          this.navStack.pushPathByName(Routes.OPEN_SOURCE, '');
        })
      Button('常见问题')
        .fontSize(ThemeTokens.FONT_SMALL)
        .fontColor(ThemeTokens.TEXT_SECONDARY)
        .backgroundColor(ThemeTokens.SURFACE_VARIANT)
        .borderRadius(ThemeTokens.RADIUS_PILL)
        .height(36)
        .layoutWeight(1)
        .onClick(() => {
          this.navStack.pushPathByName(Routes.FAQ, '');
        })
    }
    .width('100%')

    Button('重新查看引导')
      .fontSize(ThemeTokens.FONT_SMALL)
      .fontColor(ThemeTokens.TEXT_SECONDARY)
      .backgroundColor(ThemeTokens.SURFACE_VARIANT)
      .borderRadius(ThemeTokens.RADIUS_PILL)
```

```
              .height(36)
              .width('100%')
              .onClick(() => {
                this.navStack.pushPathByName(Routes.ONBOARDING, '');
              })

            Text('MoyiOhos · 阶段2 · 蓝白阅读')
              .fontSize(10)
              .fontColor(ThemeTokens.TEXT_TERTIARY)
              .width('100%')
              .textAlign(TextAlign.Center)
              .margin({ top: 6, bottom: 12 })
          }
          .width(this.isWide() ? 760 : '100%')
          .padding({
            left: ThemeTokens.PAGE_PADDING,
            right: ThemeTokens.PAGE_PADDING,
            top: 4,
            bottom: 24
          })
          .alignItems(HorizontalAlign.Start)
        }
        .width('100%')
        .justifyContent(FlexAlign.Center)
        .alignItems(VerticalAlign.Top)
      }
      .scrollBar(BarState.Off)
      .width('100%')
      .height('100%')
    }
    .title('关于', {
      systemMaterial: new uiMaterial.ImmersiveMaterial({
        style: uiMaterial.ImmersiveStyle.ULTRA_THIN,
        colorInvert: true,
        interactive: true,
        lightEffect: {}
      })
    })
    .backgroundColor(ThemeTokens.SURFACE)
  }
}
// ==================== 文件: entry/src/main/ets/pages/ApiConfigPage.ets ====================
import fs from '@ohos.file.fs';
import { uiMaterial } from '@kit.ArkUI';
import { ThemeTokens, WindowSizeClass, observeBreakpoint, Log } from 'commons';
import { EngineType, OpenAiConfig, TranslateRequest } from 'coretranslate';
import { AppEnv } from '../common/AppEnv';
import { Routes } from '../common/Routes';
import { DownloadRepository } from '../data/DownloadRepository';
import { SettingsKeys, SettingsRepository } from '../data/SettingsRepository';
```

```
const KEY_ALIAS_OPENAI = SettingsKeys.OPENAI_KEY_ALIAS;
const TAG = 'ApiConfigPage';

/** OpenAI 兼容厂商预设。 */
class ProviderPreset {
  id: string;
  name: string;
  baseUrl: string;
  model: string;
  desc: string;

  constructor(id: string, name: string, baseUrl: string, model: string, desc: string) {
    this.id = id;
    this.name = name;
    this.baseUrl = baseUrl;
    this.model = model;
    this.desc = desc;
  }
}

/** 引擎状态徽标。 */
class EngineStatus {
  text: string;
  color: string;

  constructor(text: string, color: string) {
    this.text = text;
    this.color = color;
  }
}

/** @Builder 按引用传递参数（引擎状态/选中态变化可刷新 UI）。 */
interface EngineBadgeParams {
  status: EngineStatus;
}

interface EngineRowParams {
  title: string;
  desc: string;
  icon: Resource;
  status: EngineStatus;
  active: boolean;
  onTap: () => void;
}

/** 设置卡片容器。 */
@Component
struct ApiCard {
  @BuilderParam content: () => void;
```

```
  build() {
    Column() {
      this.content()
    }
    .width('100%')
    .padding(ThemeTokens.CARD_PADDING)
    .borderRadius(ThemeTokens.RADIUS_CARD)
    .backgroundColor(ThemeTokens.CARD_BG)
    .alignItems(HorizontalAlign.Start)
  }
}

/**
 * API 配置页（S107）：引擎列表（必应 / OpenAI 兼容 / 本地 llama）+ 厂商预设、
 * Base URL / 模型 / Key 掩码配置弹窗、连通性测试与状态徽标。
 * Key 存 AssetKeyStore（alias=openai_api_key，与 translate-dev 一致，掩码不回显）。
 */
@Component
export struct ApiConfigPage {
  @State engineType: number = 0;
  @State provider: string = 'openai';
  @State baseUrl: string = '';
  @State model: string = '';
  @State hasKey: boolean = false;
  @State lowSecurity: boolean = false;
  @State showConfig: boolean = false;
  @State draftProvider: string = 'openai';
  @State draftBaseUrl: string = '';
  @State draftModel: string = '';
  @State draftKey: string = '';
  @State testing: boolean = false;
  @State testOk: boolean = false;
  @State testMessage: string = '';
  @State testingEngine: boolean = false;
  @State engineTestOk: boolean = false;
  @State engineTestMessage: string = '';
  @State localModelLabel: string = '';
  @State localModelReady: boolean = false;
  @State hint: string = '';
  @State sizeClass: WindowSizeClass = WindowSizeClass.SM;
  private navStack: NavPathStack = AppEnv.getInstance().navStack;
  private stopBreakpoint: (() => void) | undefined = undefined;
  private providers: Array<ProviderPreset> = [
    new ProviderPreset('openai', 'OpenAI', 'https://api.openai.com/v1', 'gpt-4o-mini', 'OpenAI
官方或同协议端点'),
    new ProviderPreset('deepseek', 'DeepSeek', 'https://api.deepseek.com/v1', 'deepseek-chat',
'深度求索，中文表现好'),
    new ProviderPreset('qwen', '通义千问', 'https://dashscope.aliyuncs.com/compatible-mode/v1',
'qwen-plus', '阿里云 DashScope 兼容模式'),
```

```
    new ProviderPreset('doubao', '豆包', 'https://ark.cn-beijing.volces.com/api/v3',
'doubao-pro-32k', '火山方舟 Ark 兼容端点'),
    new ProviderPreset('zhipu', '智谱 GLM', 'https://open.bigmodel.cn/api/paas/v4', 'glm-4-flash',
 '智谱开放平台'),
    new ProviderPreset('custom', '自定义', '', '', '任意 OpenAI 兼容 Base URL 与模型名')
  ];

  aboutToAppear(): void {
    this.stopBreakpoint = observeBreakpoint((c: WindowSizeClass) => {
      this.sizeClass = c;
    });
    this.load();
  }

  aboutToDisappear(): void {
    if (this.stopBreakpoint !== undefined) {
      this.stopBreakpoint();
      this.stopBreakpoint = undefined;
    }
  }

  private async load(): Promise<void> {
    this.engineType = await SettingsRepository.getNumber(SettingsKeys.ENGINE_TYPE, 0);
    this.provider = await SettingsRepository.getString(SettingsKeys.API_PROVIDER, 'openai');
    this.baseUrl = await SettingsRepository.getString(SettingsKeys.OPENAI_BASE_URL,
'https://api.openai.com/v1');
    this.model = await SettingsRepository.getString(SettingsKeys.OPENAI_MODEL, '');
    const key = await AppEnv.getInstance().getKeyStore().load(KEY_ALIAS_OPENAI);
    this.hasKey = key !== undefined && key.length > 0;
    this.lowSecurity = AppEnv.getInstance().getKeyStore().isLowSecurityMode();
    await this.detectLocalModel();
  }

  /** 检测本地 GGUF 是否已下载/导入（只读 DownloadRepository）。 */
  private async detectLocalModel(): Promise<void> {
    try {
      await AppEnv.ready();
      const tasks = await DownloadRepository.listAll();
      for (const task of tasks) {
        const path = task.localPath ?? '';
        if (path.length === 0) {
          continue;
        }
        let exists = false;
        try {
          exists = fs.accessSync(path);
        } catch (e) {
          exists = false;
        }
        if (exists) {
```

```
          this.localModelLabel = task.displayName.length > 0 ? task.displayName : path;
          this.localModelReady = true;
          return;
        }
      }
      this.localModelLabel = '';
      this.localModelReady = false;
    } catch (e) {
      this.localModelLabel = '';
      this.localModelReady = false;
    }
  }

  private flash(text: string): void {
    this.hint = text;
    setTimeout(() => {
      if (this.hint === text) {
        this.hint = '';
      }
    }, 1600);
  }

  private bingStatus(): EngineStatus {
    return new EngineStatus('免 Key · 在线', ThemeTokens.SUCCESS);
  }

  private openAiStatus(): EngineStatus {
    if (this.baseUrl.length === 0 || this.model.length === 0) {
      return new EngineStatus('未配置', ThemeTokens.WARN);
    }
    if (!this.hasKey) {
      return new EngineStatus('缺 API Key', ThemeTokens.WARN);
    }
    return new EngineStatus('已配置', ThemeTokens.SUCCESS);
  }

  private localStatus(): EngineStatus {
    if (this.localModelReady) {
      return new EngineStatus('模型就绪', ThemeTokens.SUCCESS);
    }
    return new EngineStatus('未下载模型', ThemeTokens.TEXT_HINT);
  }

  private providerName(id: string): string {
    for (const p of this.providers) {
      if (p.id === id) {
        return p.name;
      }
    }
    if (id === 'bing') {
```

```
      return '未选择厂商';
    }
    return '自定义';
  }

  /** 应用已持久化的 OpenAI 配置到运行时引擎（取消/测试后回滚用）。 */
  private async applySavedConfig(): Promise<void> {
    const base = await SettingsRepository.getString(SettingsKeys.OPENAI_BASE_URL, '');
    const model = await SettingsRepository.getString(SettingsKeys.OPENAI_MODEL, '');
    const key = await AppEnv.getInstance().getKeyStore().load(KEY_ALIAS_OPENAI);
    const translator = AppEnv.getInstance().getOpenAiTranslator();
    if (translator !== undefined) {
      translator.setConfig(new OpenAiConfig(base, key ?? '', model));
    }
  }

  /** 取消配置弹窗：把测试时写入运行时的草稿配置回滚为已保存配置。 */
  private async cancelConfig(): Promise<void> {
    await this.applySavedConfig();
    this.showConfig = false;
  }

  /** 测试当前默认引擎连通性（必应实译一词 / OpenAI 兼容列模型 / 本地查模型文件）。 */
  private async testCurrent(): Promise<void> {
    if (this.testingEngine) {
      return;
    }
    this.testingEngine = true;
    try {
      await AppEnv.ready();
    } catch (ignore) {
      // 引擎/数据库未就绪时继续走下面的失败分支
    }
    this.engineTestMessage = '';
    this.engineTestOk = false;
    try {
      if (this.engineType === 0) {
        const translator = AppEnv.getInstance().engineRegistry.get(EngineType.BING);
        const req: TranslateRequest = { text: 'hello', from: 'en', to: 'zh' };
        const result = await translator.translate(req);
        this.engineTestOk = true;
        this.engineTestMessage = `必应连通：hello → ${result}`;
      } else if (this.engineType === 1) {
        await this.applySavedConfig();
        const translator = AppEnv.getInstance().getOpenAiTranslator();
        if (translator === undefined) {
          throw new Error('OpenAI 兼容引擎未注册');
        }
        const models = await translator.listModels();
        this.engineTestOk = true;
```

```
        this.engineTestMessage = `OpenAI 兼容连通：端点返回 ${models.length} 个模型`;
      } else {
        if (this.localModelReady) {
          this.engineTestOk = true;
          this.engineTestMessage = `本地模型已就绪：${this.localModelLabel}`;
        } else {
          throw new Error('尚未下载本地 GGUF 模型');
        }
      }
    } catch (e) {
      const err = e as Error;
      this.engineTestOk = false;
      this.engineTestMessage = `测试失败：${err !== undefined && err.message !== undefined ?
err.message : e}`;
      Log.warn(TAG, `testCurrent failed: ${this.engineTestMessage}`);
    }
    this.testingEngine = false;
  }

  private async selectBing(): Promise<void> {
    this.engineType = 0;
    await SettingsRepository.set(SettingsKeys.ENGINE_TYPE, 0);
    await SettingsRepository.set(SettingsKeys.API_PROVIDER, 'bing');
    this.flash('默认引擎已切换为必应');
  }

  private async selectLocal(): Promise<void> {
    this.engineType = 2;
    await SettingsRepository.set(SettingsKeys.ENGINE_TYPE, 2);
    this.flash(this.localModelReady ? '默认引擎已切换为本地 llama' : '请先在模型管理下载 GGUF 模型');
  }

  private openConfigDialog(): void {
    this.draftProvider = this.provider === 'bing' ? 'openai' : this.provider;
    this.draftBaseUrl = this.baseUrl;
    this.draftModel = this.model;
    this.draftKey = '';
    this.testMessage = '';
    this.testOk = false;
    this.showConfig = true;
  }

  private pickProvider(id: string): void {
    this.draftProvider = id;
    for (const p of this.providers) {
      if (p.id === id) {
        if (id !== 'custom') {
          this.draftBaseUrl = p.baseUrl;
          this.draftModel = p.model;
        }
```

```
        return;
      }
    }
  }

  /** 用当前草稿 + 已保存 Key 测试 OpenAI 兼容端点。 */
  private async testConnection(): Promise<void> {
    if (this.testing) {
      return;
    }
    this.testing = true;
    this.testMessage = '';
    this.testOk = false;
    try {
      await AppEnv.ready();
      const translator = AppEnv.getInstance().getOpenAiTranslator();
      if (translator === undefined) {
        throw new Error('OpenAI 兼容引擎未注册');
      }
      let key = this.draftKey.trim();
      if (key.length === 0) {
        const saved = await AppEnv.getInstance().getKeyStore().load(KEY_ALIAS_OPENAI);
        key = saved ?? '';
      }
      if (this.draftBaseUrl.trim().length === 0) {
        throw new Error('请填写 Base URL');
      }
      if (key.length === 0) {
        throw new Error('请填写 API Key');
      }
      translator.setConfig(new OpenAiConfig(this.draftBaseUrl.trim(), key,
this.draftModel.trim()));
      const models = await translator.listModels();
      this.testOk = true;
      this.testMessage = models.length > 0
        ? `连接成功: 端点返回 ${models.length} 个模型`
        : '连接成功: 端点可达（未返回模型列表）';
    } catch (e) {
      const err = e as Error;
      this.testOk = false;
      this.testMessage = `连接失败: ${err !== undefined && err.message !== undefined ? err.message :
e}`;
      Log.warn(TAG, `test connection failed: ${this.testMessage}`);
    }
    this.testing = false;
  }

  private async saveConfig(): Promise<void> {
    try {
      const base = this.draftBaseUrl.trim();
```

```
      const model = this.draftModel.trim();
      if (base.length === 0) {
        this.testMessage = '请填写 Base URL';
        return;
      }
      if (model.length === 0) {
        this.testMessage = '请填写模型名';
        return;
      }
      if (this.draftKey.trim().length > 0) {
        await AppEnv.getInstance().getKeyStore().save(KEY_ALIAS_OPENAI, this.draftKey.trim());
      }
      await SettingsRepository.set(SettingsKeys.OPENAI_BASE_URL, base);
      await SettingsRepository.set(SettingsKeys.OPENAI_MODEL, model);
      await SettingsRepository.set(SettingsKeys.API_PROVIDER, this.draftProvider);
      await SettingsRepository.set(SettingsKeys.ENGINE_TYPE, 1);
      // 同步运行时引擎实例。
      const translator = AppEnv.getInstance().getOpenAiTranslator();
      const savedKey = await AppEnv.getInstance().getKeyStore().load(KEY_ALIAS_OPENAI);
      if (translator !== undefined) {
        translator.setConfig(new OpenAiConfig(base, savedKey ?? '', model));
      }
      this.engineType = 1;
      this.provider = this.draftProvider;
      this.baseUrl = base;
      this.model = model;
      this.hasKey = savedKey !== undefined && savedKey.length > 0;
      this.showConfig = false;
      this.flash(`已保存 ${this.providerName(this.draftProvider)} 配置`);
    } catch (e) {
      this.testMessage = `保存失败: ${e}`;
    }
  }

  private isWide(): boolean {
    return this.sizeClass === WindowSizeClass.LG;
  }

  @Builder
  statusBadge(pp: EngineBadgeParams) {
    Text(pp.status.text)
      .fontSize(ThemeTokens.FONT_SMALL)
      .fontColor(pp.status.color)
      .padding({ left: 8, right: 8, top: 3, bottom: 3 })
      .borderRadius(ThemeTokens.RADIUS_PILL)
      .backgroundColor(ThemeTokens.SURFACE_VARIANT)
  }

  @Builder
  engineRow(pp: EngineRowParams) {
```

```
    Row({ space: 12 }) {
      Column() {
        Image(pp.icon)
          .width(20)
          .height(20)
          .fillColor(pp.active ? ThemeTokens.PRIMARY : ThemeTokens.TEXT_SECONDARY)
      }
      .width(38)
      .height(38)
      .borderRadius(ThemeTokens.RADIUS_SMALL)
      .backgroundColor(ThemeTokens.SURFACE_VARIANT)
      .justifyContent(FlexAlign.Center)

      Column({ space: 3 }) {
        Row({ space: 8 }) {
          Text(pp.title)
            .fontSize(ThemeTokens.FONT_CAPTION)
            .fontWeight(FontWeight.Medium)
            .fontColor(ThemeTokens.TEXT_PRIMARY)
          if (pp.active) {
            Text('当前默认')
              .fontSize(10)
              .fontColor(ThemeTokens.PRIMARY)
              .padding({ left: 6, right: 6, top: 1, bottom: 1 })
              .borderRadius(ThemeTokens.RADIUS_PILL)
              .backgroundColor(ThemeTokens.SURFACE_VARIANT)
          }
        }
        Text(pp.desc)
          .fontSize(ThemeTokens.FONT_SMALL)
          .fontColor(ThemeTokens.TEXT_HINT)
          .lineHeight(16)
          .maxLines(3)
      }
      .alignItems(HorizontalAlign.Start)
      .layoutWeight(1)

      this.statusBadge({ status: pp.status })
    }
    .width('100%')
    .padding({ top: 10, bottom: 10 })
    .alignItems(VerticalAlign.Center)
    .borderRadius(ThemeTokens.RADIUS_SMALL)
    .onClick(() => {
      pp.onTap();
    })
  }

  @Builder
  configDialog() {
```

```
      const httpRequest = http.createHttp();
      resp = await httpRequest.request(url, options);
      httpRequest.destroy();
    } catch (e) {
      throw this.classifyHttpError(e);
    }
    if (resp.responseCode >= 400) {
      throw this.httpError(resp.responseCode, resp.result as string);
    }
    const parsed = JSON.parse(resp.result as string) as Record<string, Object>;
    const data = parsed['data'] as Array<Record<string, Object>>;
    const models: Array<string> = [];
    if (data !== undefined && data !== null) {
      for (const item of data) {
        const id = item['id'] as string;
        if (id !== undefined && id !== null && id.length > 0) {
          models.push(id);
        }
      }
    }
    return models;
  }

  private requireConfig(): OpenAiConfig {
    if (this.config.apiKey.trim().length === 0) {
      throw new TranslateError(ErrorKind.BAD_CONFIG, '请先在引擎配置中填写 API Key');
    }
    if (this.config.model.trim().length === 0) {
      throw new TranslateError(ErrorKind.BAD_CONFIG, '请先在引擎配置中选择或填写模型名');
    }
    if (this.config.baseUrl.trim().length === 0) {
      throw new TranslateError(ErrorKind.BAD_CONFIG, '请先在引擎配置中填写 Base URL');
    }
    return this.config;
  }

  private chatEndpoint(baseUrl: string): string {
    const trimmed = baseUrl.trim();
    if (trimmed.endsWith('/chat/completions')) {
      return trimmed;
    }
    return trimmed.endsWith('/') ? `${trimmed}chat/completions` : `${trimmed}/chat/completions`;
  }

  /**
   * MVP-1 prompt (kept verbatim so page translation outcomes do not change).
   * Stage-2 text translation always goes through complete() + PromptTemplates.
   */
  private legacyMessages(req: TranslateRequest, model: string, thinking: ThinkingMode):
Array<ChatMessage> {
```

```
    const toLang = this.langName(req.to);
    const fromLang = this.langName(req.from);
    const system: ChatMessage = systemMessage(
      `You are a professional translation engine. Translate the user's text into ${toLang}.
Output ONLY the translated text, without any explanation or quotes.`);
    const user: ChatMessage = userMessage(req.from === 'auto'
      ? req.text
      : `Translate the following ${fromLang} text into ${toLang}:\n${req.text}`);
    const out: Array<ChatMessage> = [system, user];
    return out;
  }

  private buildBody(messages: Array<ChatMessage>, opts: CompletionOptions, stream: boolean):
string {
    const body: ChatCompletionBody = {
      model: this.config.model,
      messages: messages,
      max_tokens: opts.maxTokens,
      temperature: opts.temperature,
      stream: stream
    };
    if (opts.thinking === ThinkingMode.ON) {
      const t: ThinkingPayload = { type: 'enabled' };
      body.thinking = t;
    } else if (opts.thinking === ThinkingMode.OFF) {
      const t: ThinkingPayload = { type: 'disabled' };
      body.thinking = t;
    }
    // AUTO deliberately sends NO thinking field: several gateways reject it.
    return JSON.stringify(body);
  }

  /**
   * POST with stream:true and incremental SSE consumption.
   * Cross-chunk UTF-8 is handled inside SseParser's Utf8ChunkDecoder, so a
   * multi-byte character split across two TCP chunks is never mangled.
   */
  private async runStream(body: string, onDelta: (delta: string) => void): Promise<string> {
    const cfg = this.requireConfig();
    const headers: Record<string, string> = {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${cfg.apiKey}`,
      'Accept': 'text/event-stream'
    };
    const options: http.HttpRequestOptions = {
      method: http.RequestMethod.POST,
      header: headers,
      extraData: body,
      connectTimeout: CONNECT_TIMEOUT_MS,
      readTimeout: READ_TIMEOUT_MS
```

```
    };
    const httpRequest = http.createHttp();
    let result = '';
    let sawSseEvent = false;
    let rawBytes: Array<number> = [];
    const parser = new SseParser({
      onEvent: (data: string) => {
        sawSseEvent = true;
        const delta = this.extractDelta(data);
        if (delta.length > 0) {
          result += delta;
          onDelta(delta);
        }
      },
      onDone: () => {
        sawSseEvent = true;
      }
    });
    httpRequest.on('dataReceive', (chunk: ArrayBuffer) => {
      const bytes = new Uint8Array(chunk);
      for (let i = 0; i < bytes.length; i++) {
        rawBytes.push(bytes[i]);
      }
      parser.push(chunk);
    });
    let streamEnded = false;
    httpRequest.on('dataEnd', () => {
      streamEnded = true;
      parser.flush();
    });
    let respCode = -1;
    try {
      // requestInStream resolves with the HTTP status code once the body ends.
      respCode = await httpRequest.requestInStream(this.chatEndpoint(cfg.baseUrl), options);
    } catch (e) {
      throw this.classifyHttpError(e);
    } finally {
      httpRequest.destroy();
    }
    if (respCode >= 400) {
      throw this.httpError(respCode, this.rawText(rawBytes));
    }
    if (result.length > 0) {
      return result;
    }
    // Server ignored stream:true (or plain JSON body): whole-body parse.
    const whole = this.rawText(rawBytes);
    if (!streamEnded) {
      parser.flush();
    }
```

```
    if (sawSseEvent && whole.trim().length === 0) {
      throw new TranslateError(ErrorKind.CONTENT, '模型未返回任何内容');
    }
    const content = this.parseCompletion(whole);
    if (content.length > 0) {
      onDelta(content);
    }
    return content;
  }

  /** Non-stream completion parse: choices[0].message.content. */
  private parseCompletion(text: string): string {
    const parsed = this.parseJson(text);
    const err = this.serverError(parsed);
    if (err !== undefined) {
      throw new TranslateError(ErrorKind.ENGINE, err);
    }
    const choices = parsed['choices'] as Array<Record<string, Object>>;
    if (choices === undefined || choices === null || choices.length === 0) {
      throw new TranslateError(ErrorKind.CONTENT, '模型未返回翻译结果');
    }
    const message = choices[0]['message'] as Record<string, Object>;
    if (message === undefined || message === null) {
      throw new TranslateError(ErrorKind.CONTENT, '模型响应缺少 message 字段');
    }
    const content = (message['content'] as string) ?? '';
    const trimmed = content.trim();
    if (trimmed.length === 0) {
      throw new TranslateError(ErrorKind.CONTENT, '模型返回了空译文');
    }
    return trimmed;
  }

  /** SSE chunk parse: choices[0].delta.content (may be absent/empty). */
  private extractDelta(data: string): string {
    const parsed = this.parseJson(data);
    const choices = parsed['choices'] as Array<Record<string, Object>>;
    if (choices === undefined || choices === null || choices.length === 0) {
      return '';
    }
    const delta = choices[0]['delta'] as Record<string, Object>;
    if (delta === undefined || delta === null) {
      return '';
    }
    // Some providers stream reasoning_content separately — it must NOT be
    // appended to the translation result.
    const content = delta['content'] as string;
    if (content === undefined || content === null) {
      return '';
    }
```

```
    return content;
  }

  private parseJson(text: string): Record<string, Object> {
    let parsed: Record<string, Object>;
    try {
      parsed = JSON.parse(text.trim()) as Record<string, Object>;
    } catch (e) {
      throw new TranslateError(ErrorKind.CONTENT, '服务端返回内容无法解析');
    }
    if (parsed === null || parsed === undefined) {
      throw new TranslateError(ErrorKind.CONTENT, '服务端返回内容无法解析');
    }
    return parsed;
  }

  /** OpenAI-compatible {"error":{"message":"..."}} extraction (upstream). */
  private serverError(parsed: Record<string, Object>): string | undefined {
    const err = parsed['error'] as Record<string, Object>;
    if (err === undefined || err === null) {
      return undefined;
    }
    const message = (err['message'] as string) ?? '';
    const type = (err['type'] as string) ?? 'unknown';
    return `OpenAI API error (${type}): ${message}`;
  }

  private async postJson(url: string, apiKey: string, body: string, timeoutMs: number):
Promise<string> {
    const headers: Record<string, string> = {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${apiKey}`
    };
    const options: http.HttpRequestOptions = {
      method: http.RequestMethod.POST,
      header: headers,
      extraData: body,
      connectTimeout: CONNECT_TIMEOUT_MS,
      readTimeout: timeoutMs,
      expectDataType: http.HttpDataType.STRING
    };
    let resp: http.HttpResponse;
    try {
      const httpRequest = http.createHttp();
      resp = await httpRequest.request(url, options);
      httpRequest.destroy();
    } catch (e) {
      throw this.classifyHttpError(e);
    }
    if (resp.responseCode >= 400) {
```

```
      throw this.httpError(resp.responseCode, resp.result as string);
    }
    return resp.result as string;
  }

  /** Classify HTTP status into readable TranslateError (US-8 场景4). */
  private httpError(code: number, body: string): TranslateError {
    let serverMessage = '';
    try {
      const parsed = JSON.parse(body) as Record<string, Object>;
      const err = parsed['error'] as Record<string, Object>;
      if (err !== undefined && err !== null) {
        const m = err['message'] as string;
        if (m !== undefined && m !== null) {
          serverMessage = m;
        }
      }
    } catch (e) {
      // Non-JSON body (gateway HTML etc.) — fall back to raw snippet.
    }
    if (serverMessage.length === 0) {
      serverMessage = body.length > 200 ? body.substring(0, 200) : body;
    }
    if (code === 401 || code === 403) {
      return new TranslateError(ErrorKind.AUTH, `鉴权失败（HTTP ${code}）：API Key
无效或无权限。${serverMessage}`);
    }
    if (code === 429) {
      return new TranslateError(ErrorKind.RATE_LIMIT, `请求过于频繁（HTTP 429）：${serverMessage}`);
    }
    if (code === 404) {
      return new TranslateError(ErrorKind.BAD_CONFIG,
        `接口不存在（HTTP 404）：请检查 Base URL 是否包含正确的 API 版本路径。/v1。${serverMessage}`);
    }
    if (code === 400) {
      return new TranslateError(ErrorKind.BAD_CONFIG, `请求被拒绝（HTTP 400）：${serverMessage}`);
    }
    if (code === 408 || code === 504) {
      return new TranslateError(ErrorKind.TIMEOUT, `服务端超时（HTTP ${code}）：${serverMessage}`);
    }
    if (code >= 500) {
      return new TranslateError(ErrorKind.ENGINE, `服务端故障（HTTP ${code}）：${serverMessage}`);
    }
    return new TranslateError(ErrorKind.ENGINE, `服务端错误（HTTP ${code}）：${serverMessage}`);
  }

  private rawBytesToText(bytes: Array<number>): Uint8Array {
    const out: Uint8Array = new Uint8Array(bytes.length);
    for (let i = 0; i < bytes.length; i++) {
      out[i] = bytes[i];
```

```
    }
    return out;
  }

  private rawText(bytes: Array<number>): string {
    if (bytes.length === 0) {
      return '';
    }
    const decoder = util.TextDecoder.create('utf-8');
    return decoder.decodeToString(this.rawBytesToText(bytes));
  }

  private langName(code: string): string {
    const map: Record<string, string> = {
      'auto': 'auto-detected language',
      'zh': 'Simplified Chinese',
      'zh-Hans': 'Simplified Chinese',
      'zh-Hant': 'Traditional Chinese',
      'ja': 'Japanese',
      'ko': 'Korean',
      'en': 'English',
      'fr': 'French',
      'de': 'German',
      'es': 'Spanish',
      'ru': 'Russian',
      'it': 'Italian',
      'pt': 'Portuguese',
      'th': 'Thai',
      'vi': 'Vietnamese',
      'id': 'Indonesian',
      'ar': 'Arabic'
    };
    const name = map[code];
    if (name !== undefined) {
      return name;
    }
    return code;
  }

  /** Map @ohos.net.http / libcurl failures into readable TranslateError kinds. */
  private classifyHttpError(e: Object): TranslateError {
    const err = e as BusinessError;
    const code: number = err.code;
    if (code === 2300028) {
      return new TranslateError(ErrorKind.TIMEOUT, '请求超时：网络较慢或服务端无响应，请重试');
    }
    if (code === 2300006 || code === 2300007 || code === 2300005) {
      return new TranslateError(ErrorKind.NETWORK, `无法连接服务器（${code}）：请检查网络与 Base URL`);
    }
    if (code === 2300060 || code === 2300058) {
```

```
      return new TranslateError(ErrorKind.NETWORK, `安全连接失败（${code}）：证书校验或代理异常`);
    }
    if (code !== undefined && code >= 2300000 && code < 2301000) {
      return new TranslateError(ErrorKind.NETWORK, `网络错误（${code}）：${err.message ?? ''}`);
    }
    return new TranslateError(ErrorKind.NETWORK, `网络请求失败：${err.message ?? '未知错误'}`);
  }
}
// ==================== 文件: coretranslate/src/main/ets/openai/OpenAiTypes.ets ====================
/**
 * Shared OpenAI-compatible chat types (stage 2 / S107).
 *
 * The translator used to build its own prompt internally and expose only
 * translate(text). Stage 2 needs the *prompt layer* to live in entry
 * (PromptTemplates: manga / novel / text / chat), so the engine now also
 * accepts an explicit messages array. These types are the contract between
 * the two layers and are intentionally free of any prompt text.
 */

/**
 * Thinking-mode tri-state (preference 'translate_thinking_mode'):
 *  · OFF  — force thinking off  → request body carries thinking:{type:'disabled'}
 *  · AUTO — follow the model default → no thinking field at all; the prompt
 *           adds a *brief*-reasoning constraint only for known reasoning models
 *  · ON   — force thinking on   → thinking:{type:'enabled'}
 */
export enum ThinkingMode {
  OFF = 0,
  AUTO = 1,
  ON = 2
}

/** Normalise anything persisted into a valid ThinkingMode. */
export function normalizeThinkingMode(raw: number): ThinkingMode {
  if (raw === ThinkingMode.OFF) {
    return ThinkingMode.OFF;
  }
  if (raw === ThinkingMode.ON) {
    return ThinkingMode.ON;
  }
  return ThinkingMode.AUTO;
}

export function thinkingModeLabel(mode: ThinkingMode): string {
  if (mode === ThinkingMode.OFF) {
    return '关闭';
  }
  if (mode === ThinkingMode.ON) {
    return '开启';
  }
```

```
  return '自动';
}

/** One chat/completions message. */
export interface ChatMessage {
  role: string;
  content: string;
}

export function makeChatMessage(role: string, content: string): ChatMessage {
  const m: ChatMessage = { role: role, content: content };
  return m;
}

export function systemMessage(content: string): ChatMessage {
  return makeChatMessage('system', content);
}

export function userMessage(content: string): ChatMessage {
  return makeChatMessage('user', content);
}

export function assistantMessage(content: string): ChatMessage {
  return makeChatMessage('assistant', content);
}

/**
 * Per-call sampling / reasoning options. Defaults mirror the MVP-1 engine
 * (temperature 0.3, 1000 tokens) so existing call sites behave identically.
 */
export class CompletionOptions {
  temperature: number = 0.3;
  maxTokens: number = 1000;
  thinking: ThinkingMode = ThinkingMode.AUTO;
}

export function makeCompletionOptions(temperature: number, maxTokens: number,
  thinking: ThinkingMode): CompletionOptions {
  const o = new CompletionOptions();
  o.temperature = clampTemperature(temperature);
  o.maxTokens = maxTokens > 0 ? maxTokens : 1000;
  o.thinking = thinking;
  return o;
}

/** Clamp temperature into the OpenAI-accepted range (0.0 — 2.0). */
export function clampTemperature(value: number): number {
  if (Number.isNaN(value)) {
    return 0.3;
  }
```

```
  if (value < 0) {
    return 0;
  }
  if (value > 2) {
    return 2;
  }
  return value;
}

/**
 * Heuristic: does the model name look like a reasoning ("thinking") model?
 * Used by AUTO mode to append a *short*-reasoning constraint, and by the UI to
 * warn that translation will be slower. Deliberately substring based: the model
 * naming space of every vendor changes faster than any allow-list.
 */
export function isReasoningModel(model: string): boolean {
  const m = model.toLowerCase();
  const hints: Array<string> = [
    '-reasoner', 'reasoner', 'reasoning', 'thinking', 'qwq', 'o1-', 'o3-', 'o4-',
    'deepseek-r1', '-r1', 'glm-z1', 'glm-z', 'seed-1-6-thinking', 'magistral'
  ];
  for (const h of hints) {
    if (m.indexOf(h) >= 0) {
      return true;
    }
  }
  return false;
}

/** Short human-readable model family for the thinking-mode hint. */
export function describeThinking(mode: ThinkingMode, model: string): string {
  if (mode === ThinkingMode.OFF) {
    return '思考模式: 关闭（最快，兼容所有模型）';
  }
  if (mode === ThinkingMode.ON) {
    return '思考模式: 开启（更准，但更慢更费 token）';
  }
  if (isReasoningModel(model)) {
    return '思考模式: 自动（推理模型，已加简短思考约束）';
  }
  return '思考模式: 自动（跟随模型默认）';
}
// ==================== 文件: coretranslate/src/main/ets/openai/SseParser.ets ====================
import { util } from '@kit.ArkTS';

/**
 * Incremental UTF-8 chunk decoder: only decodes COMPLETE byte sequences,
 * keeping a split multi-byte tail for the next chunk (plan: SSE with
 * cross-chunk UTF-8 safety). Unit-tested via entry/src/test.
 */
```

```
export class Utf8ChunkDecoder {
  private pending: Array<number> = [];
  private decoder: util.TextDecoder = util.TextDecoder.create('utf-8');

  /** Feed a chunk; returns the safely-decodable text prefix. */
  push(chunk: ArrayBuffer): string {
    const bytes = new Uint8Array(chunk);
    for (let i = 0; i < bytes.length; i++) {
      this.pending.push(bytes[i]);
    }
    let complete = 0;
    let i = 0;
    while (i < this.pending.length) {
      const b = this.pending[i];
      let len = 1;
      if (b >= 0xF0 && b <= 0xF7) {
        len = 4;
      } else if (b >= 0xE0 && b <= 0xEF) {
        len = 3;
      } else if (b >= 0xC0 && b <= 0xDF) {
        len = 2;
      } else if (b >= 0x80 && b <= 0xBF) {
        // Stray continuation byte: treat as one unit to avoid stalls.
        len = 1;
      }
      if (i + len > this.pending.length) {
        break;
      }
      i += len;
      complete = i;
    }
    if (complete === 0) {
      return '';
    }
    const ready: Uint8Array = new Uint8Array(complete);
    for (let k = 0; k < complete; k++) {
      ready[k] = this.pending[k];
    }
    const rest: Array<number> = [];
    for (let k = complete; k < this.pending.length; k++) {
      rest.push(this.pending[k]);
    }
    this.pending = rest;
    return this.decoder.decodeToString(ready);
  }

  /** Decode any remaining bytes (end of stream). */
  flush(): string {
    if (this.pending.length === 0) {
      return '';
```

```
    }
    const ready: Uint8Array = new Uint8Array(this.pending.length);
    for (let k = 0; k < this.pending.length; k++) {
      ready[k] = this.pending[k];
    }
    this.pending = [];
    return this.decoder.decodeToString(ready);
  }
}

/** SSE consumer callbacks. */
export interface SseCallbacks {
  /** One SSE event data payload (prefix stripped). */
  onEvent: (data: string) => void;
  /** Terminal [DONE] marker received. */
  onDone: () => void;
}

/**
 * Server-Sent Events stream parser: buffers raw chunks, splits frames on
 * blank lines, extracts `data:` lines, fires onEvent per event and onDone
 * on `data: [DONE]` (plan D6 / unit tests).
 */
export class SseParser {
  private decoder: Utf8ChunkDecoder = new Utf8ChunkDecoder();
  private textBuffer: string = '';
  private dataLines: Array<string> = [];
  private readonly callbacks: SseCallbacks;

  constructor(callbacks: SseCallbacks) {
    this.callbacks = callbacks;
  }

  /** Feed a raw network chunk. */
  push(chunk: ArrayBuffer): void {
    this.textBuffer += this.decoder.push(chunk);
    this.processBuffer();
  }

  /** End of stream: flush decoders and any pending event. */
  flush(): void {
    this.textBuffer += this.decoder.flush();
    if (this.textBuffer.length > 0) {
      this.consumeLine(this.textBuffer);
      this.textBuffer = '';
    }
    this.emitEvent();
  }

  private processBuffer(): void {
```

```
    let idx = this.textBuffer.indexOf('\n');
    while (idx >= 0) {
      let line = this.textBuffer.substring(0, idx);
      this.textBuffer = this.textBuffer.substring(idx + 1);
      if (line.endsWith('\r')) {
        line = line.substring(0, line.length - 1);
      }
      this.consumeLine(line);
      idx = this.textBuffer.indexOf('\n');
    }
  }

  private consumeLine(line: string): void {
    if (line.length === 0) {
      this.emitEvent();
      return;
    }
    if (line.startsWith('data:')) {
      let data = line.substring(5);
      if (data.startsWith(' ')) {
        data = data.substring(1);
      }
      this.dataLines.push(data);
    }
    // Non-data lines (event:, id:, comments) are ignored.
  }

  private emitEvent(): void {
    if (this.dataLines.length === 0) {
      return;
    }
    const data = this.dataLines.join('\n');
    this.dataLines = [];
    if (data.trim() === '[DONE]') {
      this.callbacks.onDone();
      return;
    }
    this.callbacks.onEvent(data);
  }
}
// ==================== 文件: entry/src/main/cpp/llamacpp_bridge.cpp ====================
// MoyiOhos llama.cpp NAPI bridge (T041 / plan D11).
// Exposes: getVersion / loadModel / generate(streaming via
// napi_threadsafe_function) / stopGeneration / unloadModel.
//
// Generation runs on a worker std::thread; each decoded token is
// marshalled back to the ArkTS thread through a threadsafe function
// (逐 token 回调). A handle is returned by loadModel and used by the
// other calls.
```

```
#include <napi/native_api.h>

#include <atomic>
#include <cstring>
#include <mutex>
#include <string>
#include <thread>
#include <unordered_map>
#include <vector>

#include "llama.h"

namespace {

struct ModelEntry {
    llama_model * model = nullptr;
    llama_context * ctx = nullptr;
    std::atomic<bool> stop{false};
    std::atomic<bool> busy{false};
};

std::mutex g_mutex;
std::unordered_map<int, ModelEntry *> g_entries;
int g_next_handle = 1;

struct GenCallbacks {
    napi_ref on_token = nullptr;
    napi_ref on_done = nullptr;
};

struct GenMsg {
    bool done = false;
    bool is_error = false;
    std::string text;
};

void finalize_callbacks(napi_env env, void * data, void * hint) {
    GenCallbacks * cbs = static_cast<GenCallbacks *>(data);
    if (cbs == nullptr) {
        return;
    }
    if (cbs->on_token != nullptr) {
        napi_delete_reference(env, cbs->on_token);
    }
    if (cbs->on_done != nullptr) {
        napi_delete_reference(env, cbs->on_done);
    }
    delete cbs;
}
```

```
void call_js(napi_env env, napi_value js_cb, void * context, void * data) {
    GenCallbacks * cbs = static_cast<GenCallbacks *>(context);
    GenMsg * msg = static_cast<GenMsg *>(data);
    if (cbs == nullptr || msg == nullptr) {
        delete msg;
        return;
    }
    napi_value undefined = nullptr;
    napi_get_undefined(env, &undefined);
    napi_value fn = nullptr;
    if (msg->done) {
        if (cbs->on_done != nullptr) {
            napi_get_reference_value(env, cbs->on_done, &fn);
        }
    } else {
        if (cbs->on_token != nullptr) {
            napi_get_reference_value(env, cbs->on_token, &fn);
        }
    }
    if (fn != nullptr) {
        napi_value arg = nullptr;
        napi_create_string_utf8(env, msg->text.c_str(), msg->text.size(), &arg);
        if (msg->done) {
            napi_value errArg = nullptr;
            napi_get_boolean(env, msg->is_error, &errArg);
            napi_value args[2] = {arg, errArg};
            napi_call_function(env, undefined, fn, 2, args, nullptr);
        } else {
            napi_call_function(env, undefined, fn, 1, &arg, nullptr);
        }
    }
    delete msg;
}

void run_generation(napi_env env, napi_threadsafe_function tsfn, ModelEntry * entry,
                    std::string prompt, int maxTokens, float temperature) {
    bool error = false;
    std::string error_text;
    const llama_vocab * vocab = llama_model_get_vocab(entry->model);

    std::vector<llama_token> tokens(prompt.size() + 8);
    int n = llama_tokenize(vocab, prompt.c_str(), static_cast<int32_t>(prompt.size()),
                           tokens.data(), static_cast<int32_t>(tokens.size()), true, true);
    if (n < 0) {
        tokens.resize(-n);
        n = llama_tokenize(vocab, prompt.c_str(), static_cast<int32_t>(prompt.size()),
                           tokens.data(), static_cast<int32_t>(tokens.size()), true, true);
    }
    if (n <= 0) {
        GenMsg * msg = new GenMsg();
```

```
        msg->done = true;
        msg->is_error = true;
        msg->text = "提示词分词失败";
        napi_call_threadsafe_function(tsfn, msg, napi_tsfn_blocking);
        napi_release_threadsafe_function(tsfn, napi_tsfn_release);
        return;
    }

    llama_sampler_chain_params sparams = llama_sampler_chain_default_params();
    llama_sampler * smpl = llama_sampler_chain_init(sparams);
    llama_sampler_chain_add(smpl, llama_sampler_init_top_k(40));
    llama_sampler_chain_add(smpl, llama_sampler_init_top_p(0.9, 1));
    llama_sampler_chain_add(smpl, llama_sampler_init_temp(temperature));
    llama_sampler_chain_add(smpl, llama_sampler_init_dist(LLAMA_DEFAULT_SEED));

    const int n_batch = 512;
    for (int i = 0; i < n; i += n_batch) {
        int count = (n - i) < n_batch ? (n - i) : n_batch;
        llama_batch batch = llama_batch_get_one(tokens.data() + i, count);
        if (llama_decode(entry->ctx, batch) != 0) {
            error = true;
            error_text = "提示词解码失败（模型或上下文不兼容）";
            break;
        }
    }

    std::string full;
    char buf[256];
    if (!error) {
        for (int produced = 0; produced < maxTokens; produced++) {
            if (entry->stop.load()) {
                break;
            }
            llama_token id = llama_sampler_sample(smpl, entry->ctx, -1);
            if (llama_vocab_is_eog(vocab, id)) {
                break;
            }
            int piece_len = llama_token_to_piece(vocab, id, buf, sizeof(buf), 0, false);
            if (piece_len > 0) {
                std::string piece(buf, piece_len);
                full += piece;
                GenMsg * msg = new GenMsg();
                msg->text = piece;
                napi_call_threadsafe_function(tsfn, msg, napi_tsfn_blocking);
            }
            llama_sampler_accept(smpl, id);
            llama_batch batch = llama_batch_get_one(&id, 1);
            if (llama_decode(entry->ctx, batch) != 0) {
                error = true;
                error_text = "推理中断（decode 失败）";
```

```
                break;
            }
        }
    }

    llama_sampler_free(smpl);
    entry->busy.store(false);

    GenMsg * done = new GenMsg();
    done->done = true;
    done->is_error = error;
    done->text = error ? error_text : full;
    napi_call_threadsafe_function(tsfn, done, napi_tsfn_blocking);
    napi_release_threadsafe_function(tsfn, napi_tsfn_release);
}

napi_value get_version(napi_env env, napi_callback_info info) {
    napi_value result = nullptr;
    const char * text = "llama.cpp (MoyiOhos NAPI bridge)";
    napi_create_string_utf8(env, text, NAPI_AUTO_LENGTH, &result);
    return result;
}

napi_value load_model(napi_env env, napi_callback_info info) {
    size_t argc = 1;
    napi_value args[1] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    if (argc < 1) {
        napi_throw_error(env, nullptr, "loadModel 需要模型路径");
        return nullptr;
    }
    char path[1024];
    size_t len = 0;
    if (napi_get_value_string_utf8(env, args[0], path, sizeof(path), &len) != napi_ok) {
        napi_throw_error(env, nullptr, "模型路径无效");
        return nullptr;
    }

    llama_backend_init();
    llama_model_params mparams = llama_model_default_params();
    llama_model * model = llama_model_load_from_file(path, mparams);
    if (model == nullptr) {
        napi_throw_error(env, nullptr, "模型加载失败：文件损坏或量化类型不受支持（含 1.25bit STQ1_0 本版暂不支持）");
        return nullptr;
    }
    llama_context_params cparams = llama_context_default_params();
    cparams.n_ctx = 2048;
    llama_context * ctx = llama_init_from_model(model, cparams);
    if (ctx == nullptr) {
        llama_model_free(model);
```

```
        napi_throw_error(env, nullptr, "上下文创建失败（内存不足或模型不兼容）");
        return nullptr;
    }

    ModelEntry * entry = new ModelEntry();
    entry->model = model;
    entry->ctx = ctx;

    std::lock_guard<std::mutex> lock(g_mutex);
    int handle = g_next_handle++;
    g_entries[handle] = entry;

    napi_value result = nullptr;
    napi_create_int32(env, handle, &result);
    return result;
}

napi_value generate(napi_env env, napi_callback_info info) {
    size_t argc = 6;
    napi_value args[6] = {nullptr, nullptr, nullptr, nullptr, nullptr, nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    if (argc < 6) {
        napi_throw_error(env, nullptr, "generate 参数不足");
        return nullptr;
    }
    int32_t handle = 0;
    napi_get_value_int32(env, args[0], &handle);

    std::string prompt;
    size_t plen = 0;
    napi_get_value_string_utf8(env, args[1], nullptr, 0, &plen);
    prompt.resize(plen);
    napi_get_value_string_utf8(env, args[1], &prompt[0], plen + 1, &plen);

    int32_t maxTokens = 128;
    napi_get_value_int32(env, args[2], &maxTokens);
    double temperature = 0.3;
    napi_get_value_double(env, args[3], &temperature);

    ModelEntry * entry = nullptr;
    {
        std::lock_guard<std::mutex> lock(g_mutex);
        auto it = g_entries.find(handle);
        if (it != g_entries.end()) {
            entry = it->second;
        }
    }
    if (entry == nullptr) {
        napi_throw_error(env, nullptr, "模型句柄无效或已卸载");
        return nullptr;
```

```
    }
    if (entry->busy.load()) {
        napi_throw_error(env, nullptr, "该模型正在生成中");
        return nullptr;
    }
    entry->busy.store(true);
    entry->stop.store(false);

    GenCallbacks * cbs = new GenCallbacks();
    napi_create_reference(env, args[4], 1, &cbs->on_token);
    napi_create_reference(env, args[5], 1, &cbs->on_done);

    napi_value resource_name = nullptr;
    napi_create_string_utf8(env, "llamacpp_generate", NAPI_AUTO_LENGTH, &resource_name);

    napi_threadsafe_function tsfn = nullptr;
    napi_status status = napi_create_threadsafe_function(
        env, nullptr, nullptr, resource_name, 0, 1, nullptr, finalize_callbacks, cbs, call_js,
&tsfn);
    if (status != napi_ok) {
        entry->busy.store(false);
        napi_delete_reference(env, cbs->on_token);
        napi_delete_reference(env, cbs->on_done);
        delete cbs;
        napi_throw_error(env, nullptr, "线程安全回调创建失败");
        return nullptr;
    }

    std::thread worker(run_generation, env, tsfn, entry, prompt, static_cast<int>(maxTokens),
                       static_cast<float>(temperature));
    worker.detach();

    napi_value undefined = nullptr;
    napi_get_undefined(env, &undefined);
    return undefined;
}

napi_value stop_generation(napi_env env, napi_callback_info info) {
    size_t argc = 1;
    napi_value args[1] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    int32_t handle = 0;
    if (argc >= 1) {
        napi_get_value_int32(env, args[0], &handle);
    }
    std::lock_guard<std::mutex> lock(g_mutex);
    auto it = g_entries.find(handle);
    if (it != g_entries.end()) {
        it->second->stop.store(true);
    }
```

```
    napi_value undefined = nullptr;
    napi_get_undefined(env, &undefined);
    return undefined;
}

napi_value unload_model(napi_env env, napi_callback_info info) {
    size_t argc = 1;
    napi_value args[1] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    int32_t handle = 0;
    if (argc >= 1) {
        napi_get_value_int32(env, args[0], &handle);
    }
    ModelEntry * entry = nullptr;
    {
        std::lock_guard<std::mutex> lock(g_mutex);
        auto it = g_entries.find(handle);
        if (it != g_entries.end()) {
            entry = it->second;
            g_entries.erase(it);
        }
    }
    if (entry != nullptr) {
        entry->stop.store(true);
        if (entry->ctx != nullptr) {
            llama_free(entry->ctx);
        }
        if (entry->model != nullptr) {
            llama_model_free(entry->model);
        }
        delete entry;
    }
    napi_value undefined = nullptr;
    napi_get_undefined(env, &undefined);
    return undefined;
}

} // namespace

EXTERN_C_START
static napi_value Init(napi_env env, napi_value exports) {
    napi_property_descriptor desc[] = {
        {"getVersion", nullptr, get_version, nullptr, nullptr, nullptr, napi_default, nullptr},
        {"loadModel", nullptr, load_model, nullptr, nullptr, nullptr, napi_default, nullptr},
        {"generate", nullptr, generate, nullptr, nullptr, nullptr, napi_default, nullptr},
        {"stopGeneration", nullptr, stop_generation, nullptr, nullptr, nullptr, napi_default,
nullptr},
        {"unloadModel", nullptr, unload_model, nullptr, nullptr, nullptr, napi_default,
nullptr},
    };
```

```
    napi_define_properties(env, exports, sizeof(desc) / sizeof(desc[0]), desc);
    return exports;
}
EXTERN_C_END

static napi_module g_llamacpp_module = {
    .nm_version = 1,
    .nm_flags = 0,
    .nm_filename = nullptr,
    .nm_register_func = Init,
    .nm_modname = "llamacpp",
    .nm_priv = nullptr,
    .reserved = {0},
};

extern "C" __attribute__((constructor)) void RegisterLlamaCppModule(void) {
    napi_module_register(&g_llamacpp_module);
}
// ==================== 文件: entry/src/main/cpp/ocr_bridge.cpp ====================
// Native async-capable PP-OCR rec bridge: ONNX Runtime C API.
// Det stays in ArkTS; this module only runs rec boxes on the calling thread,
// so ArkTS can invoke it from TaskPool/Worker without JS ORT session issues.

#include <napi/native_api.h>
#include "onnxruntime_c_api.h"

#include <algorithm>
#include <cmath>
#include <cstring>
#include <mutex>
#include <string>
#include <vector>

namespace {

const OrtApi * g_api = nullptr;
OrtEnv * g_env = nullptr;
OrtSession * g_session = nullptr;
OrtSession * g_det_session = nullptr;
OrtSessionOptions * g_options = nullptr;
OrtMemoryInfo * g_memory = nullptr;
OrtAllocator * g_allocator = nullptr;
std::mutex g_mutex;
std::string g_input_name;
std::string g_output_name;
std::string g_det_input_name;
std::string g_det_output_name;

bool ensure_api() {
    if (g_api == nullptr) {
```

```
        g_api = OrtGetApiBase()->GetApi(ORT_API_VERSION);
    }
    return g_api != nullptr;
}

bool ensure_env() {
    if (!ensure_api()) return false;
    if (g_env == nullptr) {
        if (g_api->CreateEnv(ORT_LOGGING_LEVEL_WARNING, "MoyiOcr", &g_env) != nullptr) {
            g_env = nullptr;
            return false;
        }
        if (g_api->GetAllocatorWithDefaultOptions(&g_allocator) != nullptr) return false;
        if (g_api->CreateCpuMemoryInfo(OrtArenaAllocator, OrtMemTypeDefault, &g_memory) !=
nullptr) return false;
    }
    return true;
}

bool init_rec(const std::string & model_path) {
    std::lock_guard<std::mutex> lock(g_mutex);
    if (!ensure_env()) return false;
    if (g_session != nullptr) {
        g_api->ReleaseSession(g_session);
        g_session = nullptr;
    }
    if (g_options == nullptr) {
        if (g_api->CreateSessionOptions(&g_options) != nullptr) return false;
        g_api->SetIntraOpNumThreads(g_options, 2);
        g_api->SetSessionGraphOptimizationLevel(g_options, ORT_ENABLE_ALL);
    }
    if (g_api->CreateSession(g_env, model_path.c_str(), g_options, &g_session) != nullptr) {
        g_session = nullptr;
        return false;
    }
    char * name = nullptr;
    if (g_api->SessionGetInputName(g_session, 0, g_allocator, &name) != nullptr || name ==
nullptr) return false;
    g_input_name = name;
    g_allocator->Free(g_allocator, name);
    name = nullptr;
    if (g_api->SessionGetOutputName(g_session, 0, g_allocator, &name) != nullptr || name ==
nullptr) return false;
    g_output_name = name;
    g_allocator->Free(g_allocator, name);
    return true;
}

inline void sample_rgb(const unsigned char * rgba, int w, int h, float fx, float fy, float
out[3]);
```

```
bool init_det(const std::string & model_path) {
    std::lock_guard<std::mutex> lock(g_mutex);
    if (!ensure_env()) return false;
    if (g_det_session != nullptr) {
        g_api->ReleaseSession(g_det_session);
        g_det_session = nullptr;
    }
    if (g_options == nullptr) {
        if (g_api->CreateSessionOptions(&g_options) != nullptr) return false;
        g_api->SetIntraOpNumThreads(g_options, 2);
        g_api->SetSessionGraphOptimizationLevel(g_options, ORT_ENABLE_ALL);
    }
    if (g_api->CreateSession(g_env, model_path.c_str(), g_options, &g_det_session) != nullptr) {
        g_det_session = nullptr;
        return false;
    }
    char * name = nullptr;
    if (g_api->SessionGetInputName(g_det_session, 0, g_allocator, &name) != nullptr || name ==
nullptr) return false;
    g_det_input_name = name;
    g_allocator->Free(g_allocator, name);
    name = nullptr;
    if (g_api->SessionGetOutputName(g_det_session, 0, g_allocator, &name) != nullptr || name ==
nullptr) return false;
    g_det_output_name = name;
    g_allocator->Free(g_allocator, name);
    return true;
}

struct DetBox { int x; int y; int w; int h; };

std::vector<DetBox> detect_boxes(const unsigned char * rgba, int w, int h) {
    std::vector<DetBox> result;
    if (g_det_session == nullptr || g_api == nullptr || w <= 0 || h <= 0) return result;
    const int DET_MAX_SIDE = 768;
    float scale = 1.0f;
    int max_side = std::max(w, h);
    if (max_side > DET_MAX_SIDE) scale = (float)DET_MAX_SIDE / (float)max_side;
    int detW = std::max(32, (int)std::round((float)w * scale / 32.0f) * 32);
    int detH = std::max(32, (int)std::round((float)h * scale / 32.0f) * 32);
    std::vector<float> input((size_t)3 * detH * detW, 0.0f);
    size_t plane = (size_t)detW * detH;
    for (int y = 0; y < detH; ++y) {
        float sy = (y + 0.5f) * h / detH - 0.5f;
        for (int x = 0; x < detW; ++x) {
            float sx = (x + 0.5f) * w / detW - 0.5f;
            float rgb[3];
            sample_rgb(rgba, w, h, sx, sy, rgb);
            size_t idx = (size_t)y * detW + x;
```

```
            input[idx] = (rgb[0] / 255.0f - 0.5f) / 0.5f;
            input[plane + idx] = (rgb[1] / 255.0f - 0.5f) / 0.5f;
            input[plane * 2 + idx] = (rgb[2] / 255.0f - 0.5f) / 0.5f;
        }
    }
    int64_t dims[4] = {1, 3, detH, detW};
    OrtValue * tensor = nullptr;
    if (g_api->CreateTensorWithDataAsOrtValue(g_memory, input.data(), input.size() *
sizeof(float),
        dims, 4, ONNX_TENSOR_ELEMENT_DATA_TYPE_FLOAT, &tensor) != nullptr) {
        return result;
    }
    const char * input_names[1] = {g_det_input_name.c_str()};
    const char * output_names[1] = {g_det_output_name.c_str()};
    OrtValue * output = nullptr;
    if (g_api->Run(g_det_session, nullptr, input_names, &tensor, 1, output_names, 1, &output) ==
 nullptr && output != nullptr) {
        OrtTensorTypeAndShapeInfo * info = nullptr;
        if (g_api->GetTensorTypeAndShape(output, &info) == nullptr && info != nullptr) {
            size_t count = 0;
            g_api->GetDimensionsCount(info, &count);
            std::vector<int64_t> shape(count);
            if (count >= 2) g_api->GetDimensions(info, shape.data(), count);
            float * data = nullptr;
            if (count >= 2 && g_api->GetTensorMutableData(output, (void **)&data) == nullptr &&
data != nullptr) {
                int predH = (int)shape[count - 2];
                int predW = (int)shape[count - 1];
                std::vector<unsigned char> mask((size_t)predW * predH, 0);
                for (size_t i = 0; i < mask.size(); ++i) mask[i] = data[i] > 0.3f ? 1 : 0;
                std::vector<unsigned char> tmp(mask.size(), 0);
                const int radius = 3;
                for (int y = 0; y < predH; ++y) {
                    for (int x = 0; x < predW; ++x) {
                        if (!mask[(size_t)y * predW + x]) continue;
                        int x0 = std::max(0, x - radius), x1 = std::min(predW - 1, x + radius);
                        for (int nx = x0; nx <= x1; ++nx) tmp[(size_t)y * predW + nx] = 1;
                    }
                }
                std::vector<unsigned char> dilated(mask.size(), 0);
                for (int y = 0; y < predH; ++y) {
                    for (int x = 0; x < predW; ++x) {
                        if (!tmp[(size_t)y * predW + x]) continue;
                        int y0 = std::max(0, y - radius), y1 = std::min(predH - 1, y + radius);
                        for (int ny = y0; ny <= y1; ++ny) dilated[(size_t)ny * predW + x] = 1;
                    }
                }
                std::vector<unsigned char> visited(dilated.size(), 0);
                std::vector<int> stack;
                float sx = (float)w / predW;
```

```
                float sy = (float)h / predH;
                for (int i = 0; i < (int)dilated.size(); ++i) {
                    if (!dilated[i] || visited[i]) continue;
                    int minX = predW, maxX = 0, minY = predH, maxY = 0, cc = 0;
                    stack.push_back(i);
                    visited[i] = 1;
                    while (!stack.empty()) {
                        int idx = stack.back();
                        stack.pop_back();
                        int x = idx % predW;
                        int y = idx / predW;
                        ++cc;
                        minX = std::min(minX, x); maxX = std::max(maxX, x);
                        minY = std::min(minY, y); maxY = std::max(maxY, y);
                        if (x > 0 && dilated[idx - 1] && !visited[idx - 1]) { visited[idx - 1] =
 1; stack.push_back(idx - 1); }
                        if (x + 1 < predW && dilated[idx + 1] && !visited[idx + 1]) {
visited[idx + 1] = 1; stack.push_back(idx + 1); }
                        if (y > 0 && dilated[idx - predW] && !visited[idx - predW]) {
visited[idx - predW] = 1; stack.push_back(idx - predW); }
                        if (y + 1 < predH && dilated[idx + predW] && !visited[idx + predW]) {
visited[idx + predW] = 1; stack.push_back(idx + predW); }
                    }
                    if (cc < 12) continue;
                    DetBox box;
                    box.x = std::max(0, (int)std::floor(minX * sx));
                    box.y = std::max(0, (int)std::floor(minY * sy));
                    box.w = std::min(w - box.x, (int)std::ceil((maxX - minX + 1) * sx));
                    box.h = std::min(h - box.y, (int)std::ceil((maxY - minY + 1) * sy));
                    if (box.w < 8 || box.h < 8) continue;
                    int ex = (int)std::round(box.w * 0.12f);
                    int ey = (int)std::round(box.h * 0.12f);
                    box.x = std::max(0, box.x - ex);
                    box.y = std::max(0, box.y - ey);
                    box.w = std::min(w - box.x, box.w + ex * 2);
                    box.h = std::min(h - box.y, box.h + ey * 2);
                    result.push_back(box);
                    if (result.size() >= 80) break;
                }
                bool changed = true;
                while (changed) {
                    changed = false;
                    for (size_t i = 0; i < result.size() && !changed; ++i) {
                        for (size_t j = i + 1; j < result.size(); ++j) {
                            const DetBox & a = result[i];
                            const DetBox & b = result[j];
                            const int gap = 8;
                            bool near = a.x - gap <= b.x + b.w && b.x - gap <= a.x + a.w
                                && a.y - gap <= b.y + b.h && b.y - gap <= a.y + a.h;
                            if (near) {
```

```
                                int x0 = std::min(a.x, b.x), y0 = std::min(a.y, b.y);
                                int x1 = std::max(a.x + a.w, b.x + b.w), y1 = std::max(a.y +
a.h, b.y + b.h);
                                result[i].x = x0; result[i].y = y0; result[i].w = x1 - x0;
result[i].h = y1 - y0;
                                result.erase(result.begin() + j);
                                changed = true;
                                break;
                            }
                        }
                    }
                }
            }
            g_api->ReleaseTensorTypeAndShapeInfo(info);
        }
    }
    if (output != nullptr) g_api->ReleaseValue(output);
    g_api->ReleaseValue(tensor);
    return result;
}

inline void sample_rgb(const unsigned char * rgba, int w, int h, float fx, float fy, float
out[3]) {
    float x = std::max(0.0f, std::min((float)w - 1.0f, fx));
    float y = std::max(0.0f, std::min((float)h - 1.0f, fy));
    int x0 = (int)std::floor(x);
    int y0 = (int)std::floor(y);
    int x1 = std::min(w - 1, x0 + 1);
    int y1 = std::min(h - 1, y0 + 1);
    float wx = x - x0;
    float wy = y - y0;
    for (int c = 0; c < 3; ++c) {
        float top = rgba[(y0 * w + x0) * 4 + c] * (1.0f - wx) + rgba[(y0 * w + x1) * 4 + c] *
wx;
        float bottom = rgba[(y1 * w + x0) * 4 + c] * (1.0f - wx) + rgba[(y1 * w + x1) * 4 + c] *
 wx;
        out[c] = top * (1.0f - wy) + bottom * wy;
    }
}

std::string rec_one(const unsigned char * rgba, int w, int h,
                    int bx, int by, int bw, int bh, const std::vector<std::string> & dict) {
    const int REC_H = 48;
    int pad = 2;
    int x0 = std::max(0, bx - pad);
    int y0 = std::max(0, by - pad);
    int x1 = std::min(w, bx + bw + pad);
    int y1 = std::min(h, by + bh + pad);
    int cw = std::max(1, x1 - x0);
    int ch = std::max(1, y1 - y0);
```

```
    int target_w = (int)std::ceil((float)REC_H * cw / ch);
    target_w = std::max(32, (int)std::ceil(target_w / 32.0f) * 32);
    target_w = std::min(target_w, 1280);
    std::vector<float> input((size_t)3 * REC_H * target_w, 0.0f);
    int used_w = (int)std::ceil((float)REC_H * cw / ch);
    for (int y = 0; y < REC_H; ++y) {
        float sy = y0 + (y + 0.5f) * ch / REC_H - 0.5f;
        for (int x = 0; x < target_w; ++x) {
            if (x >= used_w) continue;
            float sx = x0 + (x + 0.5f) * cw / used_w - 0.5f;
            float rgb[3];
            sample_rgb(rgba, w, h, sx, sy, rgb);
            size_t base = (size_t)y * target_w + x;
            input[base] = (rgb[0] / 255.0f - 0.5f) / 0.5f;
            input[REC_H * target_w + base] = (rgb[1] / 255.0f - 0.5f) / 0.5f;
            input[2 * REC_H * target_w + base] = (rgb[2] / 255.0f - 0.5f) / 0.5f;
        }
    }
    int64_t dims[4] = {1, 3, REC_H, target_w};
    OrtValue * tensor = nullptr;
    if (g_api->CreateTensorWithDataAsOrtValue(g_memory, input.data(), input.size() *
sizeof(float),
        dims, 4, ONNX_TENSOR_ELEMENT_DATA_TYPE_FLOAT, &tensor) != nullptr) {
        return "";
    }
    const char * input_names[1] = {g_input_name.c_str()};
    const char * output_names[1] = {g_output_name.c_str()};
    OrtValue * output = nullptr;
    std::string text;
    if (g_api->Run(g_session, nullptr, input_names, &tensor, 1, output_names, 1, &output) ==
nullptr && output != nullptr) {
        OrtTensorTypeAndShapeInfo * info = nullptr;
        if (g_api->GetTensorTypeAndShape(output, &info) == nullptr && info != nullptr) {
            size_t count = 0;
            g_api->GetDimensionsCount(info, &count);
            std::vector<int64_t> shape(count);
            if (count > 0) g_api->GetDimensions(info, shape.data(), count);
            float * data = nullptr;
            if (g_api->GetTensorMutableData(output, (void **)&data) == nullptr && data !=
nullptr && count >= 2) {
                int seq = (int)shape[count - 2];
                int classes = (int)shape[count - 1];
                int prev = -1;
                for (int t = 0; t < seq; ++t) {
                    int best = 0;
                    float prob = data[(size_t)t * classes];
                    for (int c = 1; c < classes; ++c) {
                        float p = data[(size_t)t * classes + c];
                        if (p > prob) { prob = p; best = c; }
                    }
```

```
                    if (best > 0 && best != prev && best < (int)dict.size()) text += dict[best];
                    prev = best;
                }
            }
            g_api->ReleaseTensorTypeAndShapeInfo(info);
        }
    }
    if (output != nullptr) g_api->ReleaseValue(output);
    g_api->ReleaseValue(tensor);
    return text;
}

napi_value init_rec_js(napi_env env, napi_callback_info info) {
    size_t argc = 1;
    napi_value args[1] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    if (argc < 1) { napi_throw_error(env, nullptr, "initRec 需要模型路径"); return nullptr; }
    char path[1024];
    size_t len = 0;
    if (napi_get_value_string_utf8(env, args[0], path, sizeof(path), &len) != napi_ok) return
nullptr;
    bool ok = init_rec(std::string(path, len));
    napi_value result = nullptr;
    napi_get_boolean(env, ok, &result);
    return result;
}

napi_value recognize_boxes(napi_env env, napi_callback_info info) {
    size_t argc = 5;
    napi_value args[5] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    if (argc < 5) { napi_throw_error(env, nullptr, "recognizeBoxes 参数不足"); return nullptr; }
    void * raw = nullptr;
    size_t raw_len = 0;
    if (napi_get_arraybuffer_info(env, args[0], &raw, &raw_len) != napi_ok) return nullptr;
    int32_t w = 0, h = 0;
    napi_get_value_int32(env, args[1], &w);
    napi_get_value_int32(env, args[2], &h);
    uint32_t box_count = 0;
    napi_get_array_length(env, args[3], &box_count);
    std::vector<int> boxes(box_count);
    for (uint32_t i = 0; i < box_count; ++i) {
        napi_value item = nullptr;
        napi_get_element(env, args[3], i, &item);
        int32_t v = 0;
        napi_get_value_int32(env, item, &v);
        boxes[i] = v;
    }
    size_t dict_len = 0;
    napi_get_value_string_utf8(env, args[4], nullptr, 0, &dict_len);
```

```
    std::string dict_text(dict_len, '\0');
    if (dict_len > 0) napi_get_value_string_utf8(env, args[4], &dict_text[0], dict_len + 1,
&dict_len);
    std::vector<std::string> dict;
    dict.push_back("");
    size_t start = 0;
    for (size_t i = 0; i <= dict_text.size(); ++i) {
        if (i == dict_text.size() || dict_text[i] == '\n') {
            std::string line = dict_text.substr(start, i - start);
            if (!line.empty() && line.back() == '\r') line.pop_back();
            if (!line.empty()) dict.push_back(line);
            start = i + 1;
        }
    }
    std::lock_guard<std::mutex> lock(g_mutex);
    napi_value arr = nullptr;
    napi_create_array_with_length(env, box_count / 4, &arr);
    if (g_session != nullptr && w > 0 && h > 0) {
        const unsigned char * rgba = (const unsigned char *)raw;
        for (uint32_t i = 0, j = 0; i + 3 < box_count; i += 4, ++j) {
            std::string text = rec_one(rgba, w, h, boxes[i], boxes[i + 1], boxes[i + 2], boxes[i
 + 3], dict);
            napi_value s = nullptr;
            napi_create_string_utf8(env, text.c_str(), text.size(), &s);
            napi_set_element(env, arr, j, s);
        }
    }
    return arr;
}

napi_value init_det_js(napi_env env, napi_callback_info info) {
    size_t argc = 1;
    napi_value args[1] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    if (argc < 1) { napi_throw_error(env, nullptr, "initDet 需要模型路径"); return nullptr; }
    char path[1024];
    size_t len = 0;
    if (napi_get_value_string_utf8(env, args[0], path, sizeof(path), &len) != napi_ok) return
nullptr;
    bool ok = init_det(std::string(path, len));
    napi_value result = nullptr;
    napi_get_boolean(env, ok, &result);
    return result;
}

napi_value detect_js(napi_env env, napi_callback_info info) {
    size_t argc = 3;
    napi_value args[3] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    if (argc < 3) { napi_throw_error(env, nullptr, "detectBoxes 参数不足"); return nullptr; }
```

```
    void * raw = nullptr;
    size_t raw_len = 0;
    if (napi_get_arraybuffer_info(env, args[0], &raw, &raw_len) != napi_ok) return nullptr;
    int32_t w = 0, h = 0;
    napi_get_value_int32(env, args[1], &w);
    napi_get_value_int32(env, args[2], &h);
    std::lock_guard<std::mutex> lock(g_mutex);
    std::vector<DetBox> boxes = detect_boxes((const unsigned char *)raw, w, h);
    napi_value arr = nullptr;
    napi_create_array_with_length(env, boxes.size() * 4, &arr);
    uint32_t k = 0;
    for (const auto & b : boxes) {
        napi_value v = nullptr;
        napi_create_int32(env, b.x, &v); napi_set_element(env, arr, k++, v);
        napi_create_int32(env, b.y, &v); napi_set_element(env, arr, k++, v);
        napi_create_int32(env, b.w, &v); napi_set_element(env, arr, k++, v);
        napi_create_int32(env, b.h, &v); napi_set_element(env, arr, k++, v);
    }
    return arr;
}

} // namespace

EXTERN_C_START
static napi_value Init(napi_env env, napi_value exports) {
    napi_property_descriptor desc[] = {
        {"initRec", nullptr, init_rec_js, nullptr, nullptr, nullptr, napi_default, nullptr},
        {"initDet", nullptr, init_det_js, nullptr, nullptr, nullptr, napi_default, nullptr},
        {"detectBoxes", nullptr, detect_js, nullptr, nullptr, nullptr, napi_default, nullptr},
        {"recognizeBoxes", nullptr, recognize_boxes, nullptr, nullptr, nullptr, napi_default,
nullptr},
    };
    napi_define_properties(env, exports, sizeof(desc) / sizeof(desc[0]), desc);
    return exports;
}
EXTERN_C_END

static napi_module g_ocr_module = {
    .nm_version = 1,
    .nm_flags = 0,
    .nm_filename = nullptr,
    .nm_register_func = Init,
    .nm_modname = "ocrasync",
    .nm_priv = nullptr,
    .reserved = {0},
};

extern "C" __attribute__((constructor)) void RegisterOcrModule(void) {
    napi_module_register(&g_ocr_module);
}
```